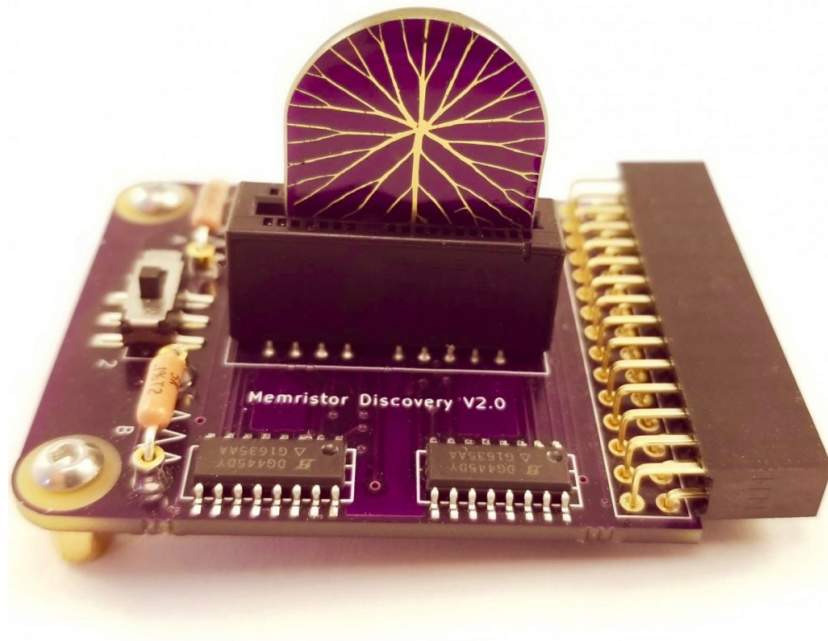


Known Memristor Discovery Manual

Alex Nugent

July 2019, Revised Sept 2020



Contents

1	Setup	3
1.1	Introduction	3
1.2	Memristor Discovery History and Purpose	3
1.3	Unboxing	4
1.4	Workstation Setup	5
1.4.1	Electrostatic Discharge	6
1.4.2	Electromagnetic Interference	8
1.4.3	Do Not Measure Resistance With A Multi-Meter!!!	8
1.5	Software Installation	9
1.5.1	Digilent Waveforms	9
1.5.2	Analog Discovery Test and Calibration	9
1.5.3	Analog Discovery 2 Calibration	11
1.5.4	Memristor Discovery Software	12
1.5.5	Memristor Discovery BoardCheck	13
2	Known Self-Directed Channel Memristors	14
2.1	Memristor Taxonomy and Structure	14
2.2	Symbol and Polarity Conventions	17
3	Memristor Discovery Board Overview	18
3.1	Version 0	18
3.2	Version 1	20
3.3	Version 2	22
4	Memristor Discovery Software Overview	23
5	DC Response	24
5.1	Setup	24
5.2	Plot Types	25
6	AC Response	31
7	Resistance Programing	34
7.0.1	Temperature Dependant Threshold	36
7.0.2	Analog Summation	37
8	Pulse Response	38
8.1	Overview	38
8.2	Read Pulse Measurement	39
8.3	Polarity Inversion due to Parasitic Capacitance	41
8.4	Pulse Programming	42
9	Thermodynamic Random Access Memory (kT-RAM)	45

10 Synapse Experiment	45
10.1 Synapse Selection	45
10.2 Synapse Instructions	47
10.3 Basic Use	47
11 Classifier Experiment	48
11.1 kT-RAM Learn Routines	49
11.2 Datasets	51
11.3 Scramble	51
11.4 Basic Use	52
11.5 A Note on Voltage Drops	54
12 Important Final Notes	54
12.1 Please Report Mistakes	54
12.2 Anomalies	55
12.3 Glass Phase Change	55
12.4 Hard Resets	55
12.5 AD2 Glitches	56
12.6 AD2 Scope and Waveform Offsets	57

1 Setup

1.1 Introduction

Welcome to the very new and exciting world of memristors! This guide will help you set up the Memristor Discovery hardware and software on your computer and guide you through some memristor experiments to help you get a better understanding of how memristors work. While the idea of a memristor has been around for many decades and many research labs around the world are constantly testing new memristive materials, Knowm's Metal-doped Self Directed Channel (M-SDC) memristors are the world's first commercially available devices. This Memristor Discovery Kit is the first commercial low-cost memristor introductory platform and currently the only way an average person can get hands-on experience with memristors. This is because Knowm M-SDC memristors are truly revolutionary. In addition to having many ideal memristor properties, they can be fabricated at high yields and have long shelf-lives. This has enabled us to move the experimental technology out of the laboratory of inventor Dr. Kris Campbell at Boise State University and into your hands. We hope you will help us to further understand this technology and build the adaptive circuits of tomorrow. Indeed, by purchasing this kit you have already helped to move the technology into the world, joining a growing community of researchers, educators, makers and inventors.

1.2 Memristor Discovery History and Purpose

We originally designed the Memristor Discovery board and software as a low cost risk mitigation step to building a Thermodynamic RAM (kT-RAM) core, a differential memristor pair "synaptic processor". Since 2015 we have been selling memristors to researchers around the world, and in a number of cases we got requests to help understand why their particular experimental setup was not working correctly. Given everybody has a unique lab setup with unique equipment, this is simply not possible for us—but we stand by our memristors and wanted to find a way that we could all 'get on the same page'. At the same time we were discouraged by the high price of most laboratory equipment. We realized that we could expand Memristor Discovery to provide a more general introductory platform for others. We decided to open-source the code and started to sell the boards, which we hand-soldered and populated one at a time as the orders came in. To our surprise, we sold more memristors in the kits than without proving the need of the platform. We have since continued to refine the software, fix bugs, add more experiments, make new board generations and write more documentation. The goal of Memristor Discovery is to provide an affordable college or advanced high school hands-on introduction to memristors. It will take you from basic DC response all the way to adaptive synapses and a learning classifier.

1.3 Unboxing

Depending on which kit you purchased, you should see items as in Figure 1. This guide can be used for all versions of the kit, which differ only slightly. The version 0.0 board contains a hard-wired memristor-resistor circuit with selectable memristors. The version 1.0 board has 1-4 Mux chips capable of dynamic routing of waveform generators and oscilloscopes as well as selectable memristors. The version 2.0 board accepts our new encapsulated chip-on-board chips with 16 memristors. The version 1.0 and 2.0 boards allow for adaptive synaptic and neuron experiments, although the version 1.0 board has been superseded with the more stable and reliable version 2.0 'common grounded anode' circuit. This manual was introduced concurrently with the release of the V2.0 boards.

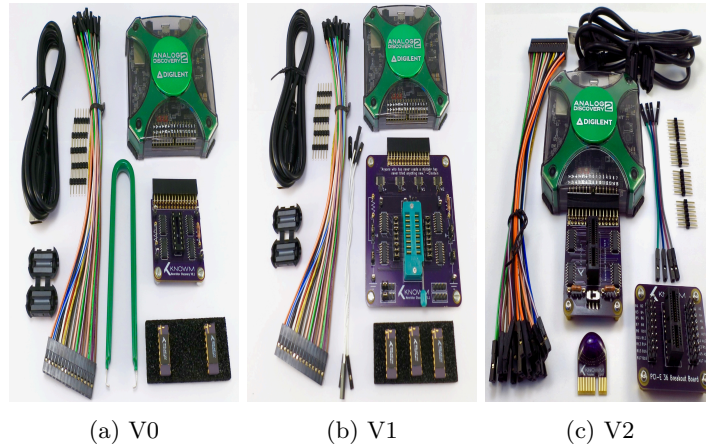


Figure 1: Memristor Discovery Kit versions with contents. Packaging has been omitted from picture.

Each V2.0 Kit should contain the following items, as seen in Figure 2

1. Analog Discovery 2 Multi Instrument Tool
2. Memristor Discovery Board V2.0
3. Knowm M-SDC Memristor 1X16 Linear Array Chip
4. Female-Female wire jumpers
5. 30 pin connector dongle
6. USB Cable Magnetic Choke
7. USB Cable
8. Male-male pin headers

9. PCI-E 36 Breakout Board

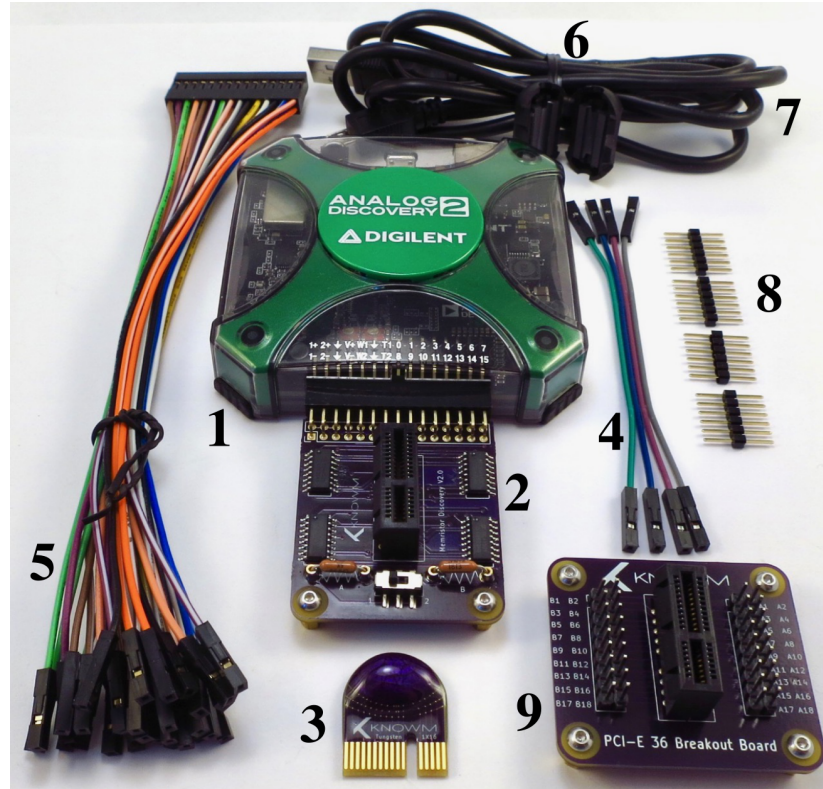


Figure 2: Memristor Discovery V2.0 Kit Contents

In addition to the above listed items, each kits contains a plastic box that comes with the Analog Discovery 2 that fits all the components.

WARNING: Do not directly touch or remove memristors from their packaging foam until your work station has been set up and you understand the basics of Electrostatic Discharge (ESD) and how to mitigate it.

1.4 Workstation Setup

You can use Memristor Discovery almost anywhere so long as you observe and understand a few things. The best location will be free from distraction, desk clutter, and electromagnetic interference. Your two main concerns are (1) Electro-Static Discharge (ESD) and (2) Electromagnetic interference. ESD is much more serious and has the potential to end the life of your memristors in a nanosecond. Electromagnetic interference can prevent your equipment from

working properly, especially during capture of fast pulses, and is more of a frustration.

1.4.1 Electrostatic Discharge

Electrostatic discharge (ESD) is the flow of electricity between two electrically charged objects caused by contact—usually *you* and *the chip you just fried*. A buildup of static electricity can be caused by number of processes, for example walking across a floor, opening a plastic bag or using equipment such as spray cleaners, heat guns, etc. To be safe you should assume that everything around you has some sort of charge on it that is waiting to equalize with other objects when they come into contact.

Static Generation Method	Volts Generated (Humidity 15%)
Walking across a carpet	35,000 V
Walking on a vinyl floor	12,000 V
Pulling paper from vinyl bag	7,000 V
Worker at bench	6,000 V

Table 1: Electrostatic generation methods and voltages.

ESD becomes especially serious if you live in a dry climate. Just take a look at Figure 1 and realize that your memristors can be fried if you apply just 1 volt without proper current limiting (to be discussed). Take ESD seriously.

The best solution is to buy or build an ESD safe workstation. This involves insuring that the surface you are working on (your desk), the thing you are working with (the chip) and yourself are wired all to the same (ground) electrical potential. You can buy dedicated equipment for this, called an anti-static mat and a wrist strap, or you can create your own solution if you know what you are doing.

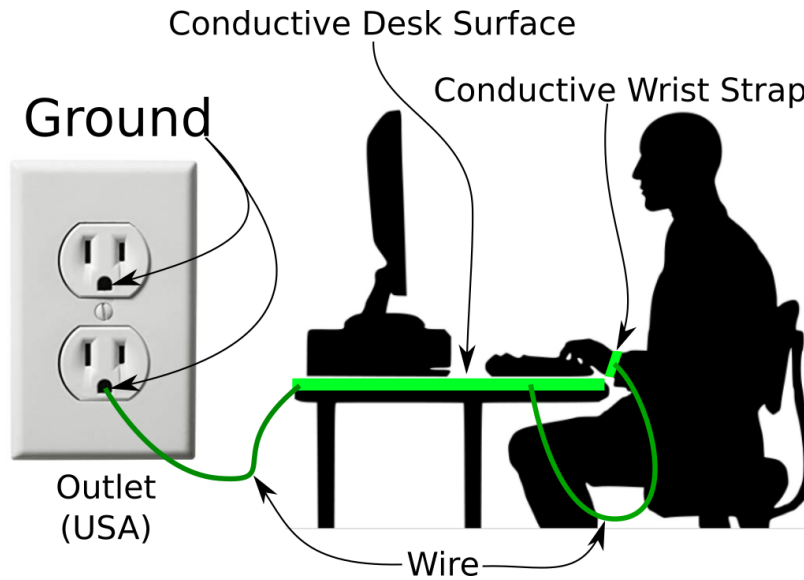


Figure 3: ESD Safe Workstation

The basics of an ESD safe workstation are shown in Figure 3. If your desk is metal and does not have paint or other coating that prevents electrical conductivity then you are almost all set—you simply need to wire yourself to your table and then wire your table to electrical ground. We will leave the details of this up to you. If you do not have an ESD safe workstation but really want to get started and are aware of the risks, we recommend the following:

1. Plug Memristor Discovery Board into the Analog Discovery 2
2. Plug the Analog Discovery 2 into your computer.
3. Make sure your computer is on.
4. Touch your finger to the left three pins between the board and the AD2 (the 1+, 2+, and Ground pins). Your are now at ground potential with all the electronics.
5. Remove the conductive black foam with Memristor chips from the bag and touch the foam against the three left pins of the AD2 as you did with your finger previously. The foam is now at ground potential.
6. Carefully place the chip into the board socket.

Anytime you are handling the memristor chips, even if at a ESD safe workstation, it is best to take care not to touch the pins as in Figure 4. When finished with the memristor chips, be sure to store them in an ESD safe configuration such as pressed (front side down) into the black conductive foam or otherwise all pins electrically shorted together.

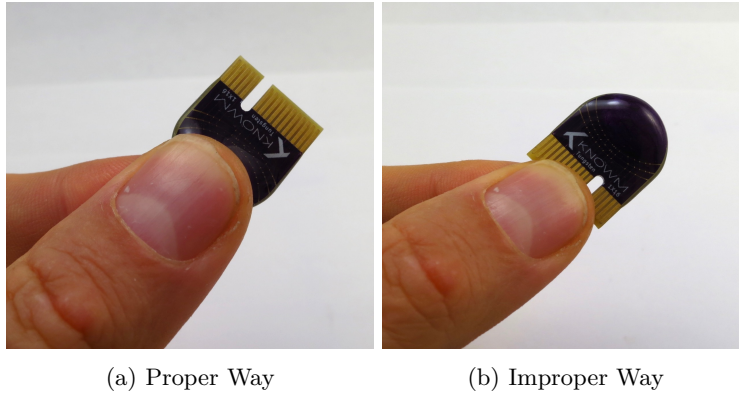


Figure 4: Proper and improper way to handle the memristor chips. In other words, try not to touch the pins.

1.4.2 Electromagnetic Interference

Electromagnetic interference is less of a concern than ESD, but it can and does cause problems. Electromagnetic radiation will induce voltage noise on metal wires that act as antennas. In all case but antennas, this is a problem. In our case it may cause a problem by introducing noise on the USB Cable. This makes it hard or impossible for the Analog Discovery 2 to communicate with your computer. There are only a few things you can do to mitigate electromagnetic interference.

1. Install the magnetic ferrite choke around your USB cable. See item 6 of Figure 2.
2. Use a shorter USB cable (also with a magnetic choke).
3. Change your work location.
4. Find the source of electromagnetic radiation and remove it.

1.4.3 Do Not Measure Resistance With A Multi-Meter!!!

A multi-meter can apply a voltage above the memristors threshold's without adequate current limiting. At best the memristor will change its conductance in response, making your measurements useless and very confusing. At worse, it will irreparably damage it via current overloading. Only apply voltages less than .1V with current limiting when measuring the resistance of a memristor.

1.5 Software Installation

1.5.1 Digilent Waveforms

Download waveforms software suitable for your operating system and computer from the Digilent website: <https://reference.digilentinc.com/reference/software/waveforms/waveforms-3/start>

Follow the instructions dependent on your specific OS:

Mac OSX Move the dwf.framework to /Library/Frameworks and Waveforms to Applications, as indicated during the install of Waveforms from the DMG.

Windows Download and run the installer.

Linux Download Waveforms .deb file. Open up the terminal and type the following commands:

```
sudo mv ~/Downloads/digilent.waveforms_3.9.1_amd64.deb /var/cache/apt/archives
cd /var/cache/apt/archives
sudo dpkg -i digilent.waveforms_3.9.1_amd64.deb

sudo mv ~/Downloads/digilent.adept.runtime_2.19.2-amd64.deb /var/cache/apt/archives
cd /var/cache/apt/archives
sudo dpkg -i digilent.adept.runtime_2.19.2-amd64.deb
```

1.5.2 Analog Discovery Test and Calibration

Once Waveforms software has been installed, open the program and verify it is functional and communicating with the AD2. We suggest you take some time to learn how to use Waveforms as it is both flexible and extensive.

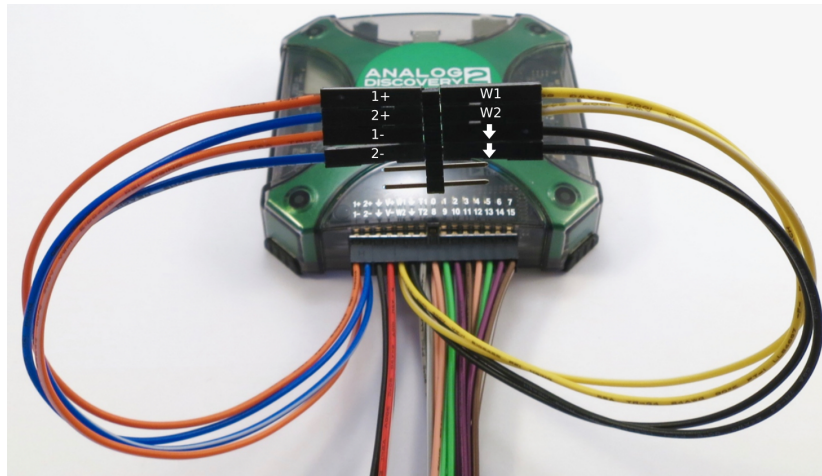


Figure 5: Waveform generator and scope self-test.

To test the AD2's waveform and scope functionality attach the 30 pin wire dongle to the AD2 and use the supplied male-male pin heads to connect waveform generator 1 to scope 1, and waveform generator 2 to scope 2 as in Figure 5. Now configure Waveforms to generate a square wave on W1 and a sine wave on W2:

1. Click the "Welcome +" button at the top of the window panel, then click on "Wavegen".
2. In the top menu bar, click on "Channels" and select "2".
3. Set the first channel to a sine wave and the second channel to a square wave.
4. click "Run" on both channels.

You should now see something like what is shown in Figure 6

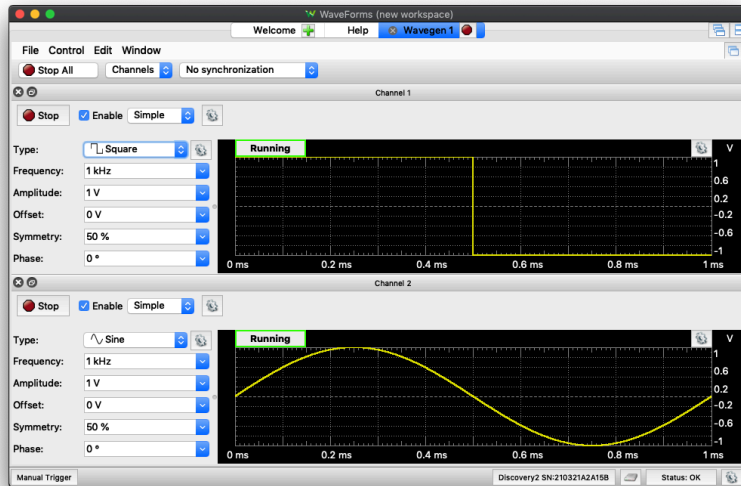


Figure 6: Waveform generators configured for self-test.

While the waveforms are running on both channels, click on the "Welcome +" button and then click on "Scope" and then "Run". If you do not see both signals, insure that each channel is selected. You should now see a square wave (yellow) and a sine wave (blue) as in Figure 7.

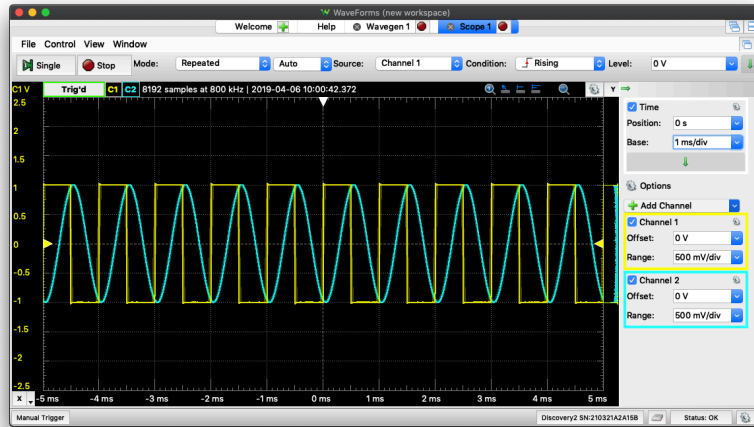


Figure 7: Square and sine waves as measured by scope 1 and scope 2.

If you do not see this, check your connections until you do.

1.5.3 Analog Discovery 2 Calibration

To insure accurate waveform generation and measurement it is advisable to calibrate your Analog Discovery 2. This will require a multi-meter for measuring voltage. In the Waveforms menu, click on "Settings" and then "Device Manager". With your device selected, click on the "Calibrate" button. You should see the following as in Figure 8.

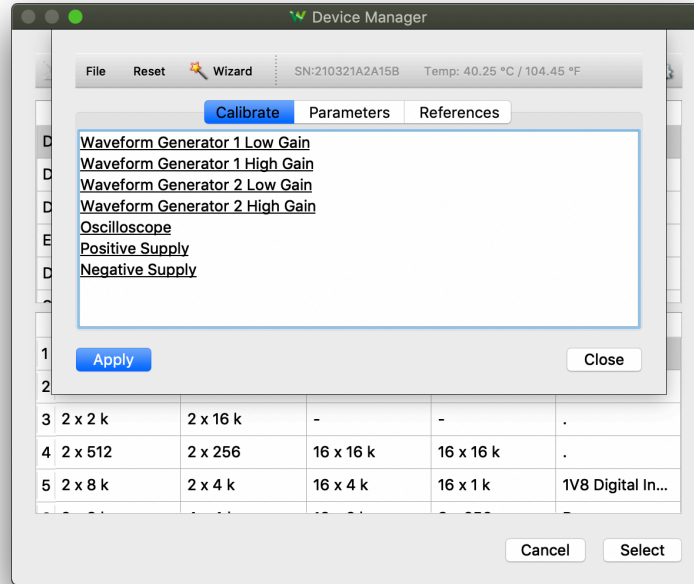


Figure 8: Analog Discovery 2 Calibration Screen

Click on each of the calibration links, starting with "Waveform Generator 1 Low Gain" and ending with "Negative Supply" and follow the on-screen instructions. When finished, click the "Apply" button and then click "Yes" when asked if you want to apply the calibration changes. While the Waveforms software resolves this through calibration, and while the calibration data is stored on the device itself, the physical AD2 unit does not actually apply the calibration. Rather, Waveforms software uses the stored parameters to correct the acquired data and generated signals. To add calibration to your measurements in Memristor Discovery, follow the instructions in the "Help" Menu in versions 2.0.1 and above.

1.5.4 Memristor Discovery Software

Memristor Discovery open-source software can be found on Github. Starting with version 0.0.8, Java is now bundled with each release and you are not required to download and install Java separately—unless you would like to develop the source code. In this case, please refer to the developer notes on the Github page.

To install the software, follow the instructions dependent on your specific OS:

Mac OSX Download the Memristor-Discovery-x.x.x.pkg file, double-click it, go through the install wizard and open the app from /Applications.

Windows Download Memristor-Discovery-x.x.x.exe file and double click the .exe file to run the install wizard.

Linux Download the Memristor-Discovery-0.0.9.deb file and from the command line run:

```
sudo mv ~/Downloads/Memristor-Discovery-0.0.9.deb /var/cache/apt/archives
cd /var/cache/apt/archives
sudo dpkg -i Memristor-Discovery-0.0.9.deb
```

1.5.5 Memristor Discovery BoardCheck

After you have installed Memristor Discovery, insure your AD2 is plugged into your computer and the Memristor Discovery board is plugged into the Analog Discovery 2. Open Memristor Discovery Software and select your board version from the "Board" menu option. Next, select the "BoardCheck" Experiment from the "Experiments" menu option.

Note: If you see a big red bar on the bottom of the window, this means that the Memristor Discovery software could not find the AD2. If all cables are attached, the issue may be FTDI drivers. Insure that the drivers have been installed, all cables are attached, and that your computer has been restarted.

Each Memristor Discovery board and chip has been tested with this program prior to shipment. It is good practice to test that everything is still operating correctly from time to time.

1-4 Mux Test (Skip if you have V0 or V2 boards). If you have the V1 board, the first test will check the four 1-4 analog multiplexers. First, click the button on the side bar labeled "1-4 Mux Board Test (V1.x)". Next, remove the right 5k Ω resistor from its socket and click the button again. This test routes the W1 and W2 signal generators to the A, B and Y nodes on the board while routing the 1+ and 2+ scopes to the same node. A specific voltage is generated and measured, and the deviation is shown. If the measured voltage deviates from the generated voltage by more than 2%, the test fails. Note that the test with the 5k Ω resistor failed, while the test without the resistor passed. This is due to impedance of the signal generators and a resulting voltage drop due to the 5k Ω resistor pulling to ground.

Switch Board Test This test checks the functionality of the four bilateral switches that are used to selectively couple/select memristors and expose them to the waveform generators. It will measure the resistance with all the switches closed and with each switch open, one at a time. Testing this properly with the V2 board is difficult unless you have a special chip that will short all terminals

of the socket. We test all boards before we ship, so it is likely your board works. In fact, if your memristor passes the discrete chip test then the switches have also passed. Note that the V2 board must be in Mode 1 to run this test.

Discrete Chip Test This test will perform an erase-read-write-read-erase-read sequence over each of the memristors, one at a time, to determine on and off resistance with the given series resistance. Note that the V2 board must be in Mode 1 to run this test.



Figure 9: Test of a 1X16 Linear Array with 5k Ω series resistor

2 Known Self-Directed Channel Memristors

2.1 Memristor Taxonomy and Structure

Known memristors are of the 'Self Directed Channel' (SDC) variety. SDC Memristors are relatively new. Just as there are multiple types of 'Oxide' and 'Phase Change' memristors, there are multiple types of SDC memristors. You can see where SDC memristors fall within a broad categorization by looking at Figure 10.

The Known M-SDC Memristors material stack relies on silver ion movement into *channels* within the active layer to change the device resistance. A metal-catalyzed reaction within the active layer generates permanent conductive channels that contain silver agglomeration sites. The amount of Ag within the channel determines the resistance of the device. Modification of the active layer with dopants enhances and tunes electrical response properties.

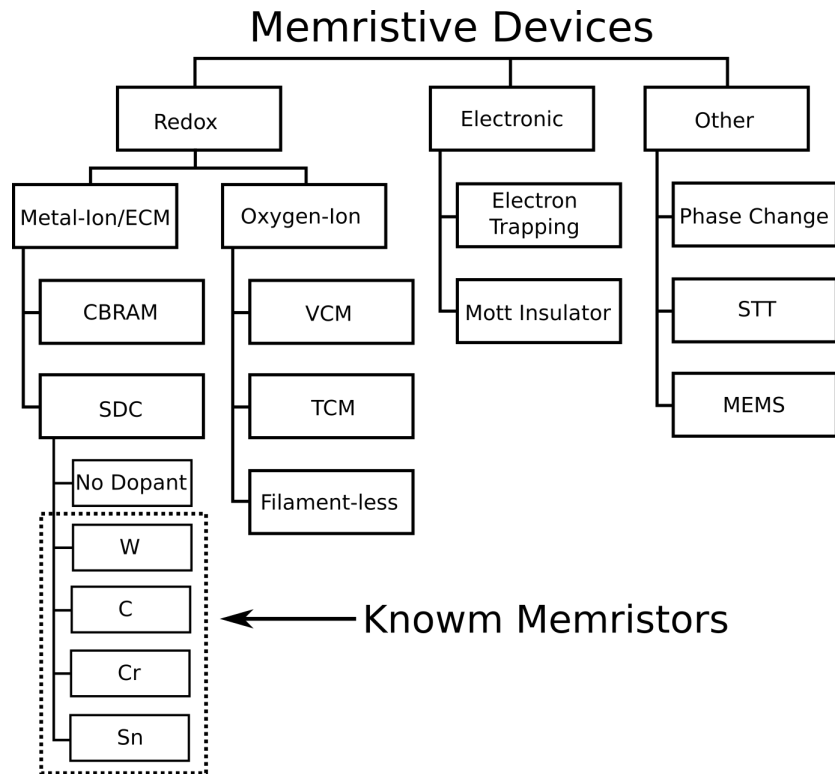


Figure 10: Taxonomy of memristors including Known M-SDC memristors

It was originally thought that metal-ion memristors could not hold analog state information (and suffered other problems), but this was turned on its head with the M-SDC memristor. While traditional metal-ion devices form (grow) metallic 'filaments' between electrodes, M-SDC devices constrain the movement of ions to a chemically created channel structure. Think of it like 'cracked glass', where silver ions move in the cracks between the electrode. The channels provide support for conductive pathways, while the addition of silver into the channels raises or lowers the conductance. The result is a much more consistent and stable device.

Known M-SDC memristors have high endurance, low switching voltages (but not too low), analog state retention, high on/off ratios and long (or potentially unlimited) shelf life.

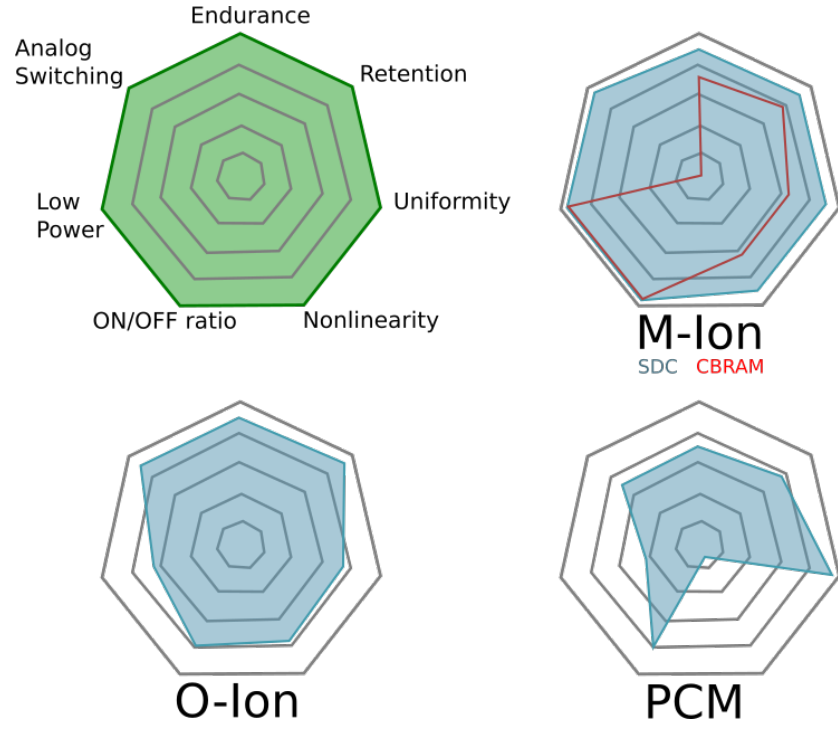


Figure 11: Comparison of memristor properties with other memristor categories

M-SDC devices are initially in a high resistance state ($M\Omega$ – $G\Omega$ range) following fabrication. The first time a device is operated after fabrication the self-directed channel is formed during application of a positive potential to the top electrode (anode). The potential required for this operation is typically the same as required during normal device operation. This first operation generates Sn ions from the SnSe layer and forces them into the active Ge_2Se_3 layer, where they undergo a chemical reaction. During this reaction, the glass network is distorted to provide conductive channels within the active layer for the movement of Ag^+ during device operation.

W	Sn	Cr	C
W	W	W	W
Ge2Se3 Adhesion	Ge2Se3 Adhesion	Ge2Se3 Adhesion	Ge2Se3 Adhesion
Ag	Ag	Ag	Ag
Ge2Se3 Mix	Ge2Se3 Mix	Ge2Se3 Mix	Ge2Se3 Mix
SnSe	SnSe	SnSe	SnSe
Ge2Se3	Ge2Se3	Ge2Se3	C+Ge2Se3
W+Ge2Se3	Sn+Ge2Se3	Cr+Ge2Se3	
Ge2Se3	Ge2Se3	Ge2Se3	
W	W	W	W

Figure 12: M-SDC material Stack. Dopants are added to the active layer to modify channel formation and effect device properties.

The resistance is tunable in the lower and higher directions by movement of Ag into or away from these channels through application of either a positive or negative potential, respectively, across the device. Knownm Memristors currently come in four variants: W, C, Sn and Cr, which refers to the dopant introduced during fabrication as in Figure 12.

2.2 Symbol and Polarity Conventions

This symbol is used internally at Knownm as it is easier to draw by hand and more accurately represents a defining characteristic of a memristor. As Leon Chua, the theoretical inventor of the memristor, has said: “If it’s pinched it’s a memristor”.



Figure 13: Knownm Memristor Symbol

A voltage applied across the device (forward voltage) with the lower-potential end on the side of the bar, will drive the device into a high conductance state, while a reverse voltage will drive the device into a less conductive state. The bar signifies the electrode adjacent to the active chalcogenide layers. In the pristine device, this is the layer furthest away from the original Ag layer. We believe the natural direction for conductance change in a memristor should be defined as increasing, as this is how most adaptive dissipative systems in nature evolve over time. By convention, a bar signifies the lower potential end in other devices like diodes. Furthermore, from an electro-chemistry merged with a semiconductor

devices perspective, we believe having the bar on the ‘cathode’ makes the most sense, since this is where reduction occurs. **Note that this is opposite to the typical memristor symbol polarity convention.**

3 Memristor Discovery Board Overview

3.1 Version 0

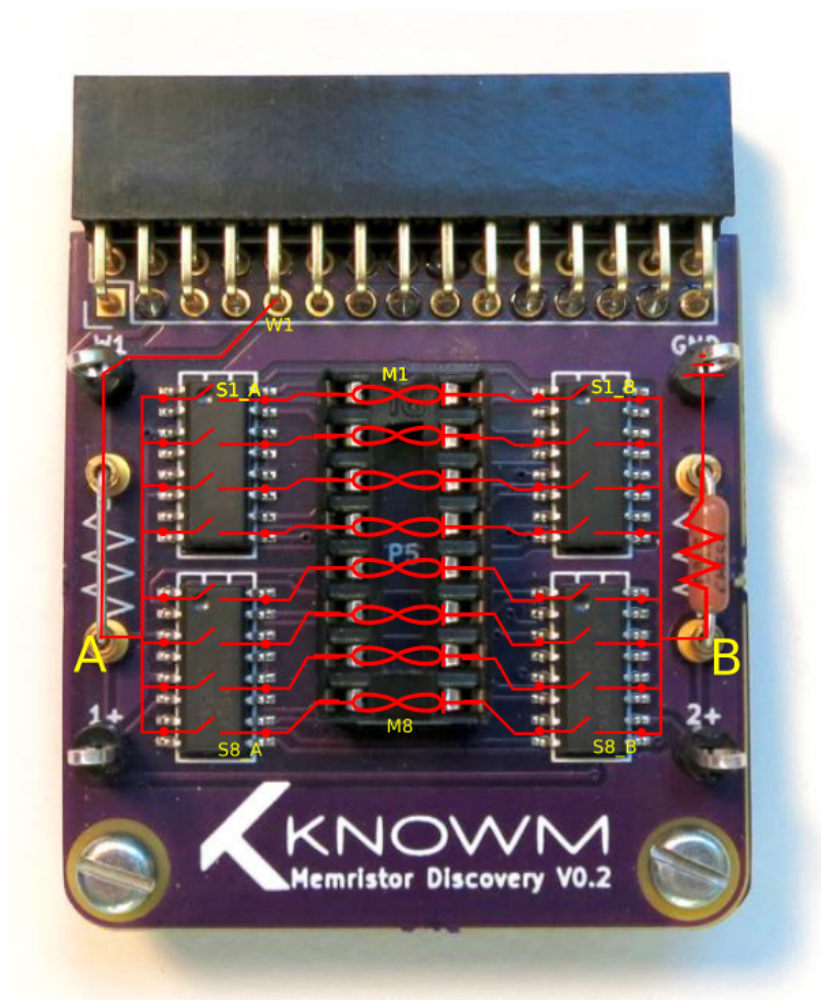
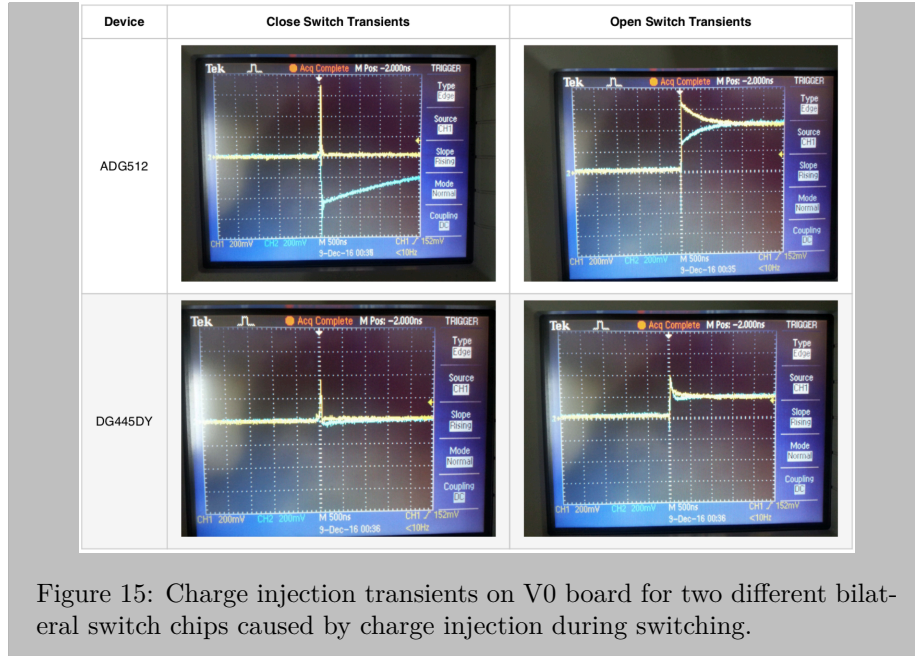


Figure 14: Overview of V0.0.X Board

The Version 0 board was designed for one purpose: to selectively couple one or more memristors (via the four bilateral switch ICs controlled by the digital

IO) to a series resistor, drive the simple circuit with waveform generator 1 and measure the voltage across the series resistor. While simple, there is a lot you can do and learn with this board, as we will guide you through a lot of it.

Pro-Tip: Charge injection and voltage transients An analog switch is formed with a NMOS and PMOS transistor in parallel. A PMOS device has about twice the area of an NMOS device so they will have about the same resistance. This imbalance of area results in twice the gate-drain capacitance for the PMOS transistor. When the switch is flipped, unequal amounts of positive and negative charge are injected onto the drain. This causes a transient voltage spike. While this is not normally a problem in many applications, its a big problem for memristors because the voltage transients will program or erase the memristors. If you are going to design a board, use chips with low charge injection or take measures to keep transients below .1V. When we first designed the board, we used Analog Device ADG512 Bilateral switches, which have a stated charge injection of 11pC. This resulted in memristor erase and programming when switches were flipped, so we found an alternate with the Vishay DG445DY, which has a stated charge injection of -1pC. In Figure 15 the voltage across the memristors is the difference between the yellow and blue trace, with .2V per block scale. Charge injection from the ADG512 chips caused over .4V transients for hundreds of nanoseconds, which is more than enough to program or erase the memristors. Transient voltages are probably one of the single most common challenge faced by folks trying to use memristors, and they are much more common than you might think. Otherwise very expensive and sophisticated equipment will generate transient voltage spikes when turned on or off or when modes are changed. In fact, this is one reason why we decided to invest more effort into the Memristor Discovery platform. We would get graduate students and professors who were using our chips and not getting the expected results—but not considering that their own equipment could be at fault. Memristor Discovery provides a common 'reference platform' so everybody can get calibrated and we and others can provide demonstrations that others can easily repeat.



3.2 Version 1

As in the V0 board, the four bi-lateral switches are used to selectively couple memristors into the circuit. We also have the same 'A' and 'B' nodes. Unlike the V0 board, we have four 1-4 analog multiplexer chips, abundant headers and jumpers, selectable resistor shunts and a floating node 'Y'. The purpose of the 1-4 Analog Muxes is to programmatically route the waveform generators (W1, W2) and oscilloscopes (1+, 2+) to one of four locations on the board: The A, B and Y nodes as well as an external pin 'E'. The V1 board was originally designed to test differential-pair memristors of various configurations, but it can be used more generally for a number of experiments by configuring jumpers. Programmatic routing of waveform generators and scopes allows for complex measurement and driving of memristor circuits.

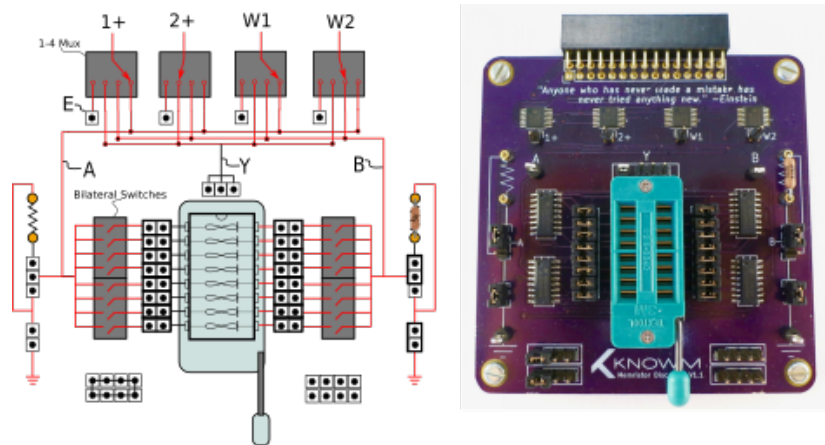


Figure 16: Overview of V0.1.X Board

3.3 Version 2

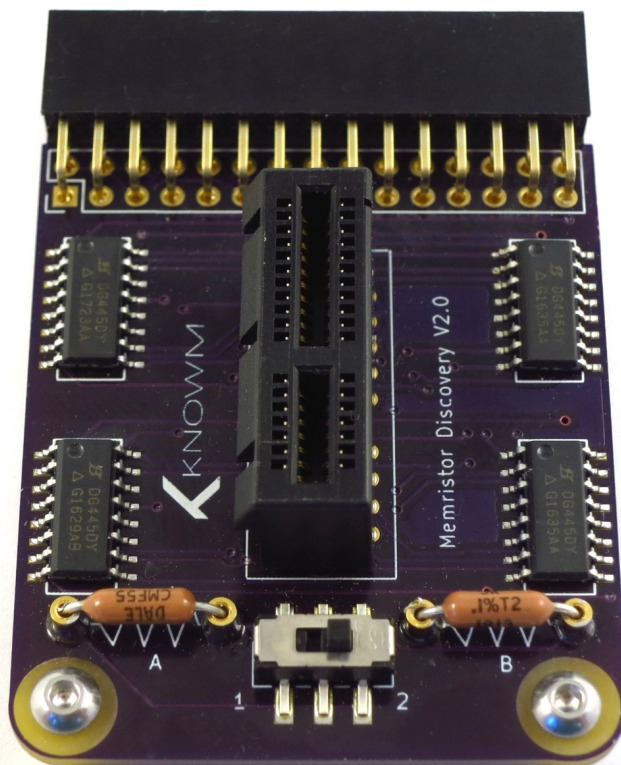


Figure 17: V2 Memristor Discovery Board

The V2 board flips the basic memristor-resistor circuit upside down and backwards, as in Figure 18 D. The board is configured to allow individual access to each of 16 memristors in a 1X16 Linear Array chip. It has two 'Modes' that can be selected by the physical switch. Mode 1 (to the left) enables individual or unitary access of memristors while Mode 2 enables differential access. In Mode 1, the series resistance is determined by both A and B resistors, so that if two $20\text{k}\Omega$ resistors are in the socket the measured resistance is $10\text{k}\Omega$. If one of the resistors is pulled, the measured resistance is $20\text{k}\Omega$. The measured resistance value should be entered into the preferences field of all experiments.

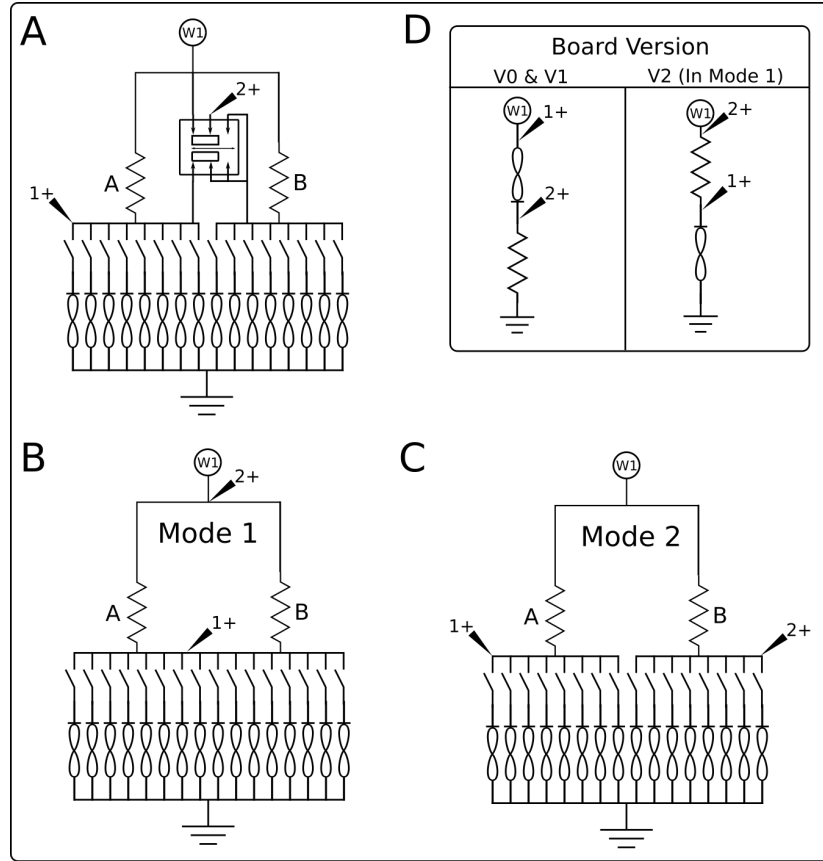


Figure 18: A) Board Circuit, B) Mode 1 selected, C) Mode 2 Selected, D) Series resistor circuit change from V0/V1 boards.

4 Memristor Discovery Software Overview

Boards and Experiments Memristor Discovery software is extensible and can support multiple board types or versions. We currently have three boards: V0, V1 and V2 and anticipate the addition of new boards and improvements to the existing boards over time. As some experiments will only work with certain boards, you must insure that the board selected in the menu matches the board you are using. Experiments can be selected in the 'Experiment' menu. We will review five experiments in this guide, from basic memristor characterization to a learning neuron circuit.

Preferences Every experiment consists of various controls that set various parameters like voltage, pulse duration, etc. Each experiment supports default settings that can be set via the preference menu. Once an experiment is selected,

click on 'Window' and select 'Preferences'. This will open a dialog box with various settings that will become the default.

Help Every experiment provides help documentation that can be viewed by clicking 'Window' and 'Help'. This is intended primarily to help set up any connections and describe the purpose of the experiment. For more extensive help, please refer to this guide.

Console Some experiments include back end processing and rely on certain conditions to be met for accurate results. To help debug problems and provide more feedback to the operator, we created a 'Console' view that allows simple messages to be printed to a scrolling dialog. The console can be viewed by clicking on 'Window' and then 'Console'.

Memristor Selection Most of the experiments will require manual selection of one or two memristors. This is done by clicking on one of the radio-buttons located at the top of the window, just beneath the main menu. If no memristor is selected the bar will turn red to warn you that no memristor is selected.

AD2 Board Connectivity All experiments require a connection to the Analog Discovery 2. If the AD2 is plugged in via the USB cable, Memristor Discovery software will connect. If connection does not occur, the lower bar on the window will turn red to indicate there is no connection. Insuring the USB cable is connected, click on the 'Board On/Off' switch at the bottom of the window.

Charts and Data Export All charts are produced with our open-source plotting package XChart. To save a picture of the plot or to export the data to a CSV file, right click on the chart and select 'Save As...' or 'Export To...', respectively.

Bug Reporting If you find a bug or would like a new feature, please go to the Memristor Discovery GitHub page and file an 'Issue':

<https://github.com/knownm/memristor-discovery>

Due to a small team and the fact that the software is run across multiple operating systems, we rely of reporting of bugs or problems to improve the platform, so please do not hesitate to give us your (constructive) feedback. Also keep in mind that Memristor Discovery is open-source. If you are a Java programmer then why not make your own contributions?

5 DC Response

5.1 Setup

Taking care to avoid static discharge, place a discrete memristor chip into the socket of the Memristor Discovery board. Open the "DC" experiment. Make

sure your V2 board is in 'Mode 1'.

The DC app allows you to drive a memristor in series with a resistor, as in Figure 18, with various ramping functions including sawtooth, sawtoothupdown, triangle and triangleupdown at time scales from 10 to 1000ms. The number of applied ramping signals can be selected and the response can be observed as either a time series (V1+ vs T and V2+ vs T), I vs V or G vs V plot, revealing the behavior of the memristor at slow timescales.

Go to the experiment preferences (Window->Preferences) and insure that the series resistor placed in the right resistor socket matches the value specified, in Ohms. We do not recommend a resistor value less than $5k\Omega$ to avoid memristor burn out if you are using the Tungsten W-SDC type memristors and $20k\Omega$ if you are using the Carbon C-SDC memristors. If you have both $5k\Omega$ resistors in each socket, just remove one of them. If you have two $20k\Omega$ resistors in each socket, the measured resistance is $10k\Omega$ —and this is the value you must enter in the control panel or preferences.

Configure the left panel controls to the following:

Waveform	TriangleUpDown
W2Amplitude[V]	.8
Period[ms]	500
Pulse Number	5

Now select the first memristor in the top bar, which will be highlighted red if no memristor is selected, and then click the "Start" button. You should see a result similar to what is shown below in the "Capture", "I-V" and "G-V" plots. If you do not see this, or what you see looks strange, select a new memristor (being careful to de-select the prior memristors), and press "Start" again. If you are using a Burn and Learn chip, there can be up to four devices that did not pass our quality control test. This could be due to a number of reasons, but in any case you should have at least four devices that will produce a result as shown.

5.2 Plot Types

Once data is captured it will be displayed in the main plot panel. Radio buttons under the plot can be used to view the data in various ways, as seen below.

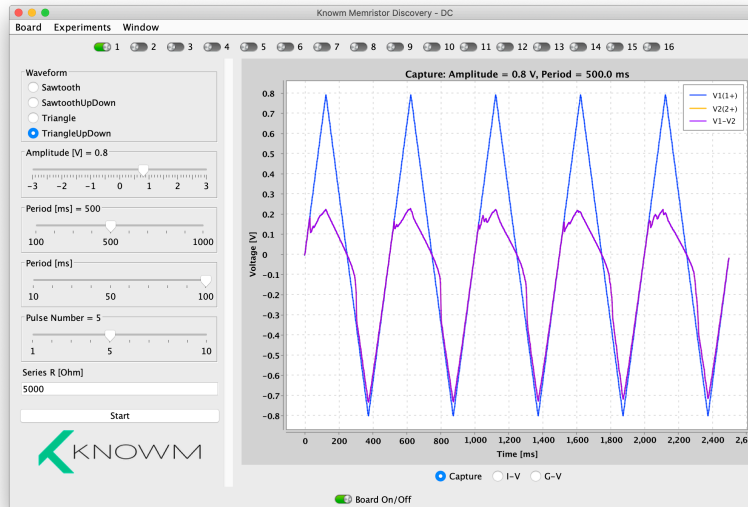


Figure 19: DC Capture of Memristor

Capture Plot Note how the voltage drop across the memristor (purple) rises along with the driving voltage, V_1 , until about .15V. After this point the voltage drop remains about the same, with some 'jagged peaks', until V_1 decreases. When the driving voltage goes negative, the voltage drop across the memristor is almost identical—indicating that the resistance of the memristor must be much larger than the series resistor. To get a better sense of what is going on here, let's look at the I-V view.

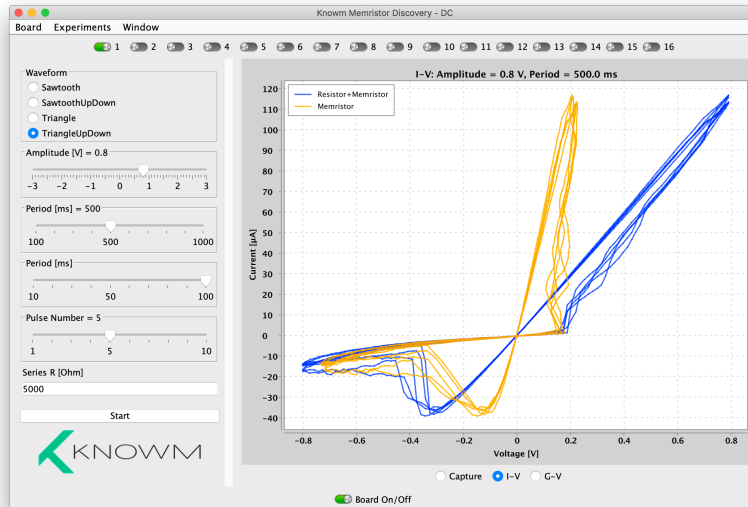


Figure 20: DC I-V plot capture of memristor

I-V Plot The I-V view give us the classical "Pinched Hysteresis Loop", which is a defining characteristic of a memristor. The I-V plot shows the driving voltage, V_1 , on the horizontal axis and the current, I , on the vertical axis. The current is computed by measuring the voltage across the known series resistor ($V=V_2-V_1$), and using Ohms Law: $I = \frac{V}{R}$. Let's progress over one cycle, using Figure 21 as a reference.

1. As the voltage is increased from zero, the resistance of the memristor is at first in the High Resistance State (HRS). The HRS is usually about $1 \text{ M}\Omega$, and the resulting current is low, probably about $1 \text{ }\mu\text{A}$.
2. As V_1 increases, the current stays low until the memristor's forward threshold, V_F is reached. At this point the current increases dramatically as the memristors resistance falls.
3. When V_1 reaches the peak amplitude of $.8\text{V}$ it starts a gradual decline and the current drops in step. Now that the memristor has decreased it's resistance the current for each applied voltage value is much higher, and so we have the first lobe of the hysteresis loop.
4. The current continues to fall to zero as V_1 goes to zero, the two 'crossing at zero'. This implies that there is no energy stored in the device, as a capacitor or inductor. A memristor stores information, however, in its change of resistance.
5. When the applied voltage goes negative, the resistance increases when the

voltage drop across the memristor exceeds it's reverse adaptation threshold, V_R , putting us back into the HRS.

This cycle repeats for as many times as we want to drive it—in this example 5 times. M-SDC memristors have rather good endurance, so you can cycle them over a billion times before noticeable degradation (this is about 31 years of switching each second), so long as you are careful about limiting the current and applied voltage.

Before we move on to the next section, lets quickly calculate the HRS and LRS by eye-balling the plot. Using Ohms Law, $R = \frac{V}{I}$, we can read the resistance as the slope of the HRS and LRS lines: $R_H RS = \frac{.2}{1 \times 10^{-6}} = 200k$ and $R_L RS = \frac{.2}{110 \times 10^{-6}} = 1.8k$. Finally, lets convert the LRS to mS by taking its inverse and multiplying by 1000: .55mS.

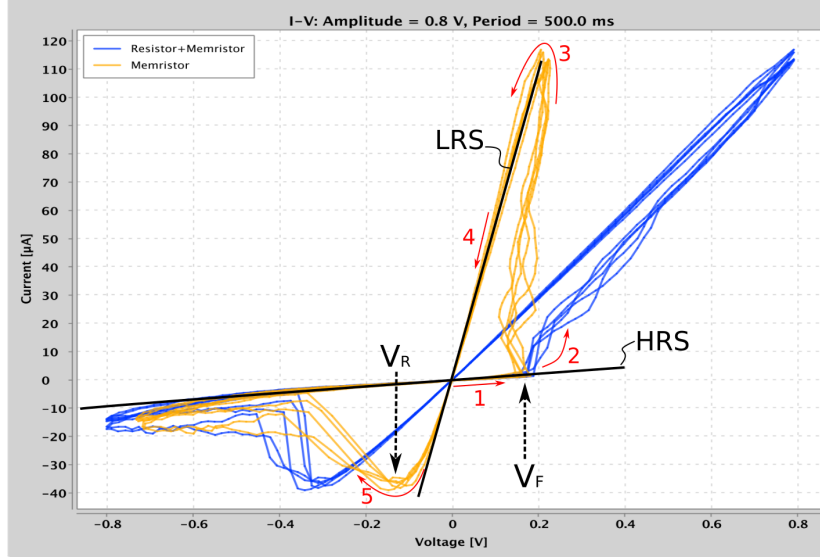


Figure 21: IV plot of memristor under sinusoidal .8V driving voltage with HRS, LRS, forward threshold (V_F) and reverse threshold (V_R) labeled

The HRS and LRS are not intrinsic properties of a memristor While it is tempting to look at 21 and declare the HRS/LRS to be 200/1.8 k Ω , in fact this is only valid *for this particular circuit under this particular drive frequency and with this particular memristors history*. That is, the HRS and LRS are not absolute properties of a memristor, but rather relative and dynamic properties since they are strongly influenced by the circuit the memristor is part of and how it is being driven. There are three primary reasons. First, the current through the memristor must be limited to protect it. This current limiting places a circuit constraint on the LRS. Without this limit the LRS would go all the way to 100's or even 10's of Ohms and burn out. Second, the HRS is

affected by the memristors history of use. The harder and longer you drive a memristor, the lower the HRS can get. A virgin device will have a resistance above $1\text{ G}\Omega$, but this will drop after it forms its first conductive channels. If you allow large currents, for example 1 mA or more, the HRS will drop to $100\text{ k}\Omega$ or lower. Third, depending on the frequency of the circuit, a memristors adaptation rate will strongly determine the HRS and LRS. You will see this for yourself in the next experiment. For these reasons it's debatable what IV curve (yellow or blue) is the 'correct' curve to show for the memristor. While the yellow trace shows the voltage drop across the memristor, *the voltage drop across the memristors is a function of the series resistor*. The memristor circuit determines the response not just the memristor in isolation. Any attempt to drive a memristor circuit in isolation will surely render the device dead due to lack of current limiting.

A History of Abuse If operated at higher currents we can cause permanent changes that lower the HRS. This is reflected by the $200\text{ k}\Omega$ HRS we just measured. So why are we doing this? The Memristor Discovery board has capacitance from the bilateral switches, the chip socket and board traces that total about 140 pF . As the resistance of the memristor and series resistor increases, this capacitance becomes a significant factor in taking measurements—as does the 1X scopes on the AD2, which represent a $1\text{ M}\Omega$ path to ground. The result is that it is harder to accurately measure high resistances on the board. If you want to keep the HRS at higher resistances, you should limit the current more by using resistors of a higher value. If you want to switch at low currents, below $1\mu\text{A}$, you may not want to use Memristor Discovery platform. You may also want to use Carbon SDC memristors instead, as they are specifically designed to be switched at low currents.

G-V Plot The G-V plot shows our data plotted as the calculated conductance versus the applied voltage. We can see clearly see the 'conductance loop'. The conductance that we calculated matches the conductance calculated at the peak voltage of the IV Plot, which is good. There are two features of the plot that we should point out. First, as we go from zero volts toward $.2\text{V}$ we see the conductance stays low until we get to forward adaptation threshold at about $.2\text{V}$. From $.3\text{V}$ to $.8\text{V}$ we see a roughly linear circuit conductance increase as the voltage is increased. So why does the conductance of the resistor+memristor circuit not just jump all the way up quickly at $.2\text{V}$? In the HRS most of the voltage drop is across the memristor, and so we see the conductance increase when we get to the forward adaptation threshold, V_F . At this point the resistance of the memristor will fall—and so will the voltage drop across it. The result is that the conductance will increase only enough so that the voltage drop across the memristor is equal to V_F . The series resistor provides negative feedback when forward biased. The more the memristor lowers its resistance, the less the voltage drop across it, requiring even more applied voltage to keep increasing

its conductance. We will use this fact in the next experiment to program the resistance of the devices.

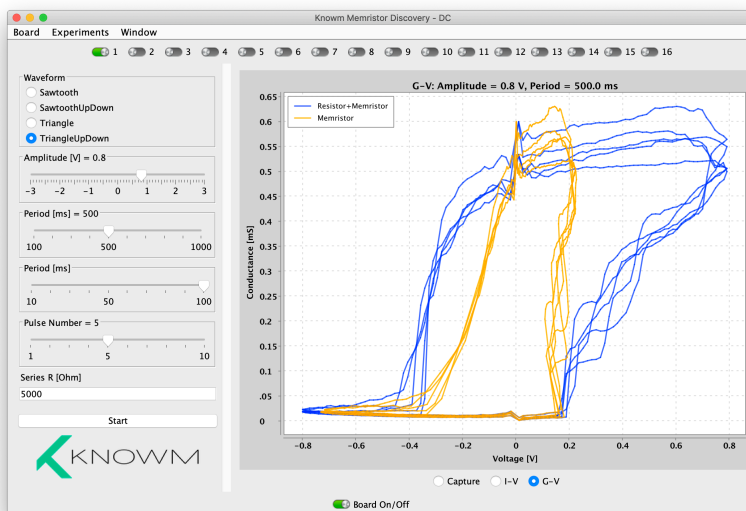


Figure 22: DC G-V plot capture of memristor

The second feature of Figure 22 to take note of is that when we get to the peak conductance and the voltage is dropping, the conductance is not perfectly stable. Rather, it falls a bit as the voltage falls. This is due to a *relaxation* of the memristor. When you drive the memristor into a high conductance state and then remove the drive voltage, it will relax slightly to a lower conductance. It only relaxes a bit—not all the way back to the HRS. This will become more obvious when we get to the Pulse Experiment. As we get close to zero ($< .05V$) the measurements can become wildly inaccurate due to noise.

Pro Tip: Use the same low magnitude voltage to measure a memristor's resistance Non-linearities in the measurement equipment, i.e. MOSFETs in switches and drivers, can lead to inconsistencies when taking measurements at different voltages. That is, the very same resistance measured with two different drive voltages can result in two different readings. Parasitic capacitance can also cause inconsistencies if you are not careful. Finally, if the voltage exceeds the memristors adaptation thresholds it will change as you are measuring it. The result is that, for accurate and consistent measurements, use a low-voltage pulse of fixed magnitude ($V < .1$) and time ($T \gg T_R C$).

Before we move on, go ahead and run a DC sweep as we just have for all the

memristors on the chip and take note of their forward and reverse thresholds. What is the variance? A DC sweep is the most reliable method to determine if you have a good, failing or non-functional device.

6 AC Response

We will now step up the frequency of the applied waveform by using the Hysteresis experiment, which allows us to apply a waveform and monitor the I-V and G-V plots in real time while we change the amplitude, frequency and offset. The circuit under investigation is the same as the DC experiment, i.e. Figure 18 D.

Select the Hysteresis experiment from the main menu and then select one of the memristors from the top radio-button menu. Leave the offset at '0'. Set the amplitude to .8V and using the lower frequency slider set the frequency to 10Hz. Finally, make sure that the Series resistor value matches the resistor that you have in the board socket. We ship boards with 5k Ω resistors, so this is the default. Below the plot check the 'I-V' radio button. Finally, click the 'Start' button. Any device that passed the DC characterization should work here, so select one (and only one) of those devices. You should see something like Figure 23.

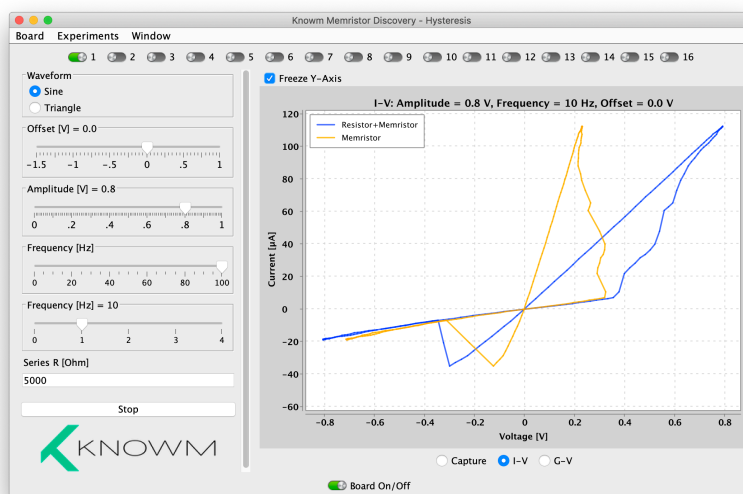


Figure 23: 10Hz I-V plot of Tungsten memristor

Note that the plot is constantly auto-sizing the Y-Axis. This is helpful, but it can also get annoying if you are not changing any of the waveform parameters. Click the square radio button in the top-left of the plot to freeze the Y-axis and prevent this. Now increase the frequency using the bottom frequency slider,

going from 10Hz to 100Hz, 1kHz and finally 10kHz. You should see something like Figure 24. Notice how the hysteresis loop changes, with the HRS and LRS changing as a function of frequency. This is a hallmark of an analog memristor, i.e. a memristor capable of holding intermediate states. Remember how we said that the HRS and LRS are not absolute properties of a memristor but also dependent on how the circuit is driven? This is your first really clear example.

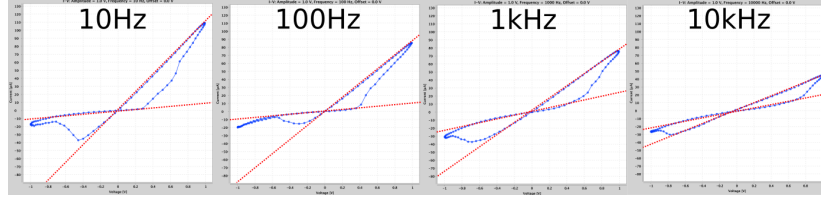


Figure 24: Comparison of 10Hz, 100Hz, 1kHz and 10kHz I-V plots of Tungsten memristor looking at the HRS and LRS in each case.

What is going on here? In each case the same voltage amplitude and waveform is applied, and yet the response of the memristor is different. This implies that the change in conductance of the memristor is not just a function of applied voltage, but also *time*. This makes sense of course, as nothing in the physical world can change immediately. When a voltage above the memristors threshold is applied, it reacts by increase or decreasing it's conductance. This takes time, and so if the drive signal changes before the memristor has finished changing, its conductance will not change by much. At some point, when the frequency has increase so much that the memristor cannot keep up, the "pinched hysteresis loop" will collapse to a line. Well, in theory that is! In the real world we also have parasitic capacitance, both in the device but also in our measurement equipment. The I-V response to a capacitor is a loop, so this is why you might not get a perfect line at higher frequencies. This effect will be more and more pronounced as you use larger series resistors. Go ahead and try it for yourself! Use a 100k Ω series resistor and replace the memristor chip with a similar value resistor and observe the I-V response at high frequencies. What you see is the parasitic capacitance of the board.

Now let's take a quick look at the G-V plot. Set the frequency to 1Hz and click on the 'G-V' radio button under the plot. You should now see two plots, one for the conductance of the memristor versus the drive voltage (Blue) and another for the conductance of the memristor versus the voltage drop across the memristor (orange). We want to point out two features of this plot.

First take a look at (1) and (2), which shows the memristor conductance versus the applied drive voltage and voltage across the memristor. Notice that once the voltage across the memristor reaches the forward threshold, the conductance increases while the voltage drop is very close to constant. Meanwhile, the drive voltage keeps increasing. This is a result of negative feedback specific to a forward-biased memristor-resistor circuit, and we explained why this occurs when discussing Figure 22. If you do not understand please take another look

at that. We are going to use this negative feedback mechanism to program the conductance of the memristor in the next section.

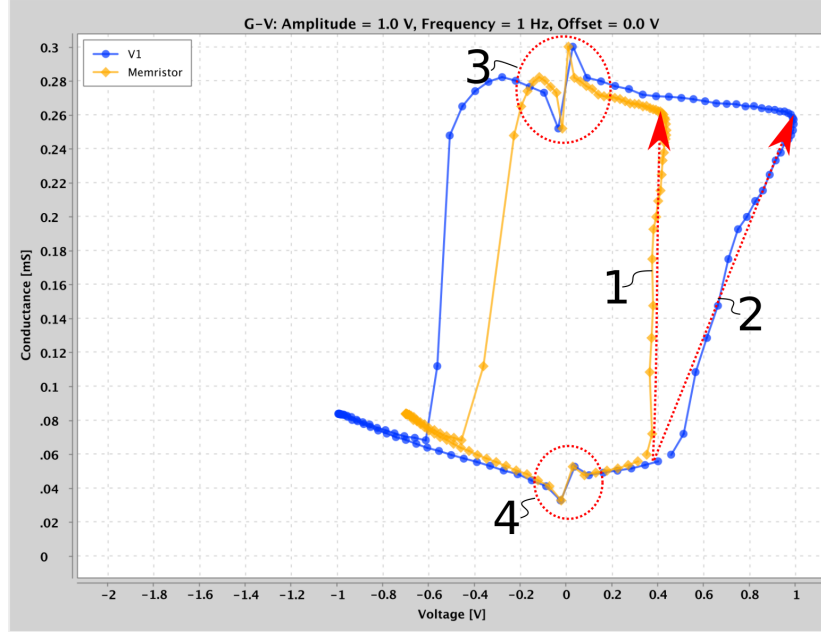


Figure 25: V-G plot of Tungsten memristor at 10Hz.

Second, take a look at how the conductance sharply rises or falls near $V = 0$, as in (3) and (4). This is a *measurement artifact*. While there are some methods to 'clean it up' algorithmically, it's important to understand because it gives you a better appreciation for the importance and limitations of measurement techniques when compared to theory. The code for computing the conductance is simple, and is as follows.

```
// HysteresisExperiment.java, ~line 260.
double[] conductance = new double[rawdata2.length];
double[] voltage = new double[rawdata1.length];

for (int i = 0; i < conductance.length; i++) {
    double I = rawdata2[i] / controlModel.getSeriesResistance();
    double G = I / (rawdata1[i] - rawdata2[i]) *
        HysteresisPreferences.CONDUCTANCE_UNIT.getDivisor();
    G = G < 0 ? 0 : G;
    double ave = (1 - resultModel.getK()) * (resultModel.getAve()) +
        resultModel.getK() * (G);
    resultModel.setAve(ave);
    conductance[i] = ave;
    voltage[i] = rawdata1[i] - rawdata2[i];
}
```

'rawdata2' is the voltage V_2 and 'rawdata1' is the voltage V_1 , as in circuit of Figure 18. The conductance is computed in two steps. First, we compute the current by taking the voltage drop across the series resistor and dividing by the stated resistance:

$$I = \frac{V_2}{R_s}$$

Next, we compute the conductance by dividing the current by the voltage drop across the memristor:

$$G = \frac{I}{V_1 - V_2}$$

If we combine these two equations together we get:

$$G = \frac{V_2}{R_s(V_1 - V_2)}$$

The measurement problem occurs as the applied voltage goes to zero. When this happens, we are strongly effected by (1) measurement resolution of the AD2, (2) measurement offsets between the 1+ and 2+ scopes, and (3) measurement noise. Noise could, when operating at very low drive voltages, cause the measured conductance to be *negative*, in which case we can be completely assured the measurement is useless. You should always understand how your measurement equipment works and, most importantly, *it's limitations and failure modes*. This is even more important when dealing with new devices like memristors, which can be very confusing before you even account for equipment and measurement complications.

7 Resistance Programing

In this section we are going to use the "Offset" slider on the Hysteresis experiment to manually control the applied voltage. With your memristor selected, set the offset to something low and positive, like .05V. Set the Amplitude to 0V. Set the frequency to 10Hz. Make sure you have the G-V plot enabled and click 'Start'. Make sure the "Freeze Y-Axis" radio button is not enabled. What you should see is a small cluster of measurements. The Y axis will give you the conductance and the X value will give you the applied voltage, which should be equal to the the offset. The chart is going to update at the rate specified by the frequency. Now, click on the 'Offset' slider to make sure it is selected and then press the 'left arrow' key on your keyboard until the offset is -.5V. Now carefully press the right-arrow key until the voltage is back at .05V. You have now reset the device. Now slowly use the right arrow key to increase the offset voltage and watch how the conductance changes. Do this to a positive voltage of .75V. Once the conductance has increased, check the 'Freeze' box to lock the axis. Now use the left arrow to lower the offset voltage back to .05V. If the measurement data goes out of the plot, un-click and then re-click the 'Freeze' check box to rescale the axis. You have now set the device and completed a programming loop. Let us go through this all in detail in reference to Figure 26.

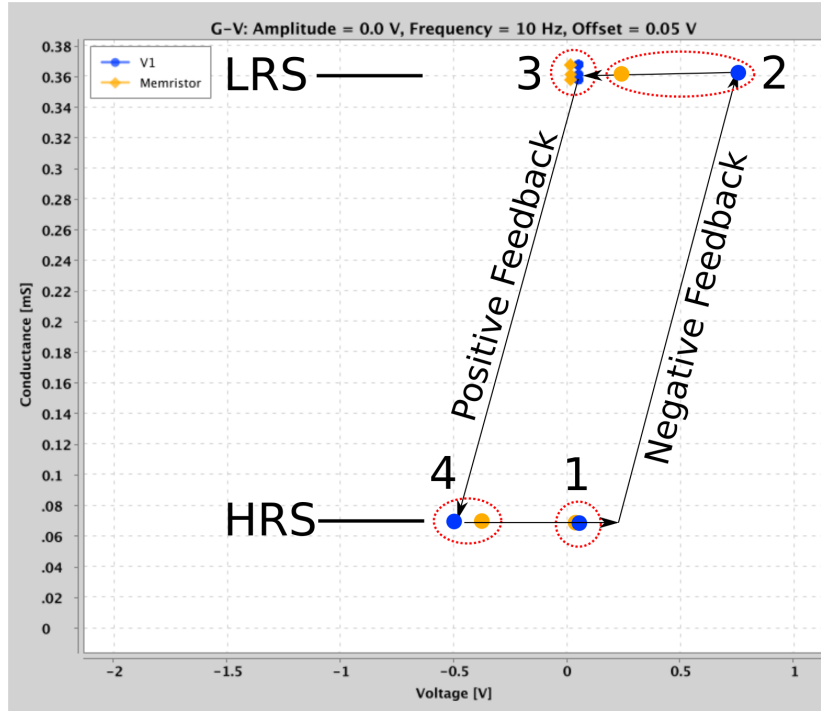


Figure 26: DC Programming Loop

We start in the HRS, as in Figure 26 (1). As we increase the applied voltage, the measured conductance will remain low until we hit the forward threshold of the device. The conductance of the device will suddenly jump but, provide you do not further increase the voltage, will remain stable. At this point, every increase in voltage will result in an increase in conductance. Due to the forward bias and series resistor, the memristor experiences negative feedback. When the conductance of the memristor increases the voltage drop across it will reduce since the total voltage drop across the memristor and the series resistor must be equal to the applied voltage. The lower voltage drop across the memristor, in turn lowers to below the threshold, and the memristor 'shuts off'. The result is that this simple circuit will convert a increase in applied voltage to increase in conductance—and prevent a run-away increase in the conductance. If you increase the voltage to .5V instead of .75V, for example, you will reach two different 'equilibrium conductances'. In this way you can program the conductance of the memristor. If you would like to program to higher resistance states, use a larger series resistor.

Note that none of this is true for a reverse biased circuit! As the applied voltage crosses the memristors reverse threshold, the conductance of the device will decrease. This will cause a larger voltage drop across the device, which will in turn cause even more conductance decrease, and so on. The result is a

positive feedback loop and a resulting 'avalanche' back to the HRS. All of this complex behavior from only a memristor in series with a resistor!

7.0.1 Temperature Dependant Threshold

As you go through the DC programming loop of Figure 26 you may notice something peculiar as you go from the HRS (1) to the LRS (2). As the voltage is increased, the memristor does not change until its threshold is hit. At this point the conductance of the memristor 'jumps' up and the voltage drop across the memristor, given by the orange dot, reduces. As the voltage continues to increase the conductance of the memristor goes up but the voltage drop across the memristor likely will not surpass the initial threshold observed in its first conductance jump. Why is this? We believe it is because the thresholds of the device lower as the temperature increases. When we start in the HRS there is very little current and consequently the device stays cool. Just before we reach the 'static threshold' (Figure 27 A), the threshold is higher due to the device being cold. Right after the static threshold is hit, the memristor heats up as current flows increase. Consequently the threshold reduces, which in turn causes the conductance to jump until the voltage drop across the memristor is equal to the 'dynamic threshold' (Figure 27 A). This effect may not be noticeable if the HRS has been reduced due to high forward currents.

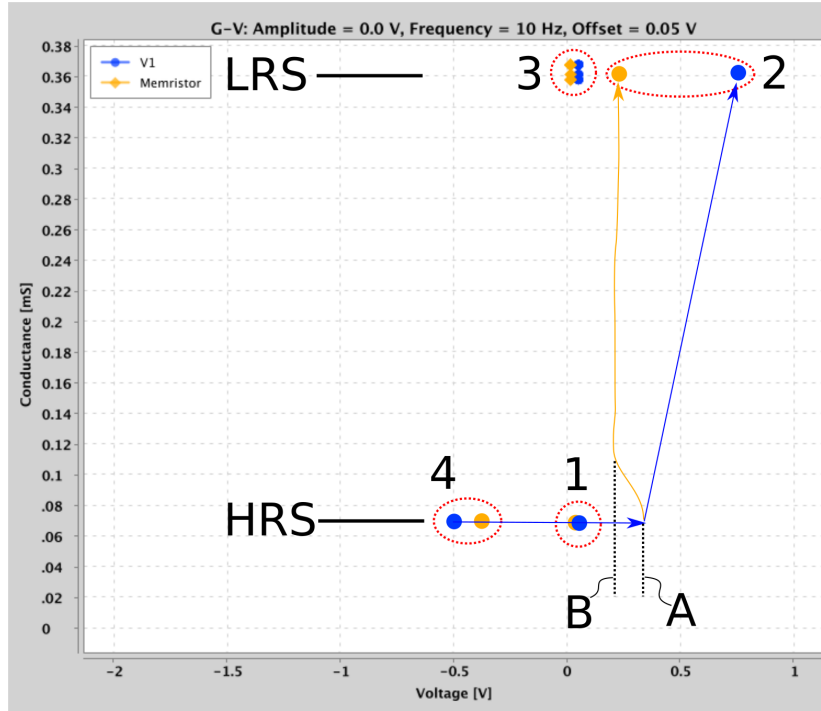


Figure 27: Dynamic (B) vs static thresholds (A) caused by temperature dependence.

7.0.2 Analog Summation

Using the method above, program the conductance of a memristor. Be sure to return the voltage to .05V when you are done. Now, de-select the memristor from the top menu. The measured conductance should fall close to zero, which is the conductance of the closed switches. Now select it again. The measured conductance should jump back up. In some cases the memristor conductance will have decayed, but it should be very close to where it was. De-select the memristor and then select a new memristor and program its conductance, being careful to return the voltage to .05V. Now, with the second memristor selected and the voltage at .05V, select your previously programmed memristor so that they are both selected. The measured conductance is the sum of the two conductances.

While a very simple demonstration, this idea is very powerful and is at the heart of most efforts to reduce neural networks to memristive circuits. We have stored two conductance values into memristors and then used the physics of circuits to add these conductances together.

Memristor Conductance Decay W-SDC Memristor are generally good at holding their conductance state, but as you program the conductance of the devices you may notice decay. In some cases this decay is very pronounced due to the device itself, and we would consider that a defective memristor (also called a 'diffusive' memristor these days). However, there are two other cases that are more common and should be expected. First, the conductance of a device will decay slightly after you remove the programming voltage, usually over a minute or two and this is normal and should be expected. Conductance decay is typically more pronounced at lower programmed conductance values. Second, and this is actually important, the act of selecting and de-selecting memristors can erase the devices due to charge injection from the switches. This effect is more pronounced as the programmed conductance of the memristor is lower and the series resistor is lower, since it takes longer for injected charge to dissipate to ground. While the bilateral switches on the Memristor Discovery board have been selected for their low charge injection, the effect has *not* been reduced to zero. For more information, be sure to read the 'ProTip' discussing charge injection above Figure 15.

8 Pulse Response

WARNING: You must exercise caution when using the Pulse Experiment! Notice how the voltage amplitude goes all the way up to 3V? Only in rare circumstances should you use this setting or you will risk a memristor burn out. Always start the pulse experiment with a low voltage ($< .5V$) pulse to check the memristor resistance before increase the magnitude. In some circumstances, if the memristor conductance is substantially lower than the series resistor, you may need to use a higher magnitude pulse to achieve a sufficient voltage drop across the memristor to make it change. Given the board parasitic capacitance, it is safer to use lower magnitude pulses with a longer duration instead of higher magnitude pulses with a short duration.

8.1 Overview

In the Pulse experiment we apply pulses of various magnitudes, durations, shapes, etc, to a memristor while we measure its resistance. We use the same basic circuit as the prior experiments (See figure 18). Pulse properties can be selected on the left controls. When a selection or change is made, a preview of the pulse shape is displayed in the main plot. When you are satisfied with your pulse (or pulse train), you can apply the pulse by clicking on the 'Start' button. Immediately after clicking the 'Start' button, the captured waveform will display in the main plot area. At the same time, a series of 'read' pulses will be fired at an interval set by the 'Sample Period [s]' value, which is set

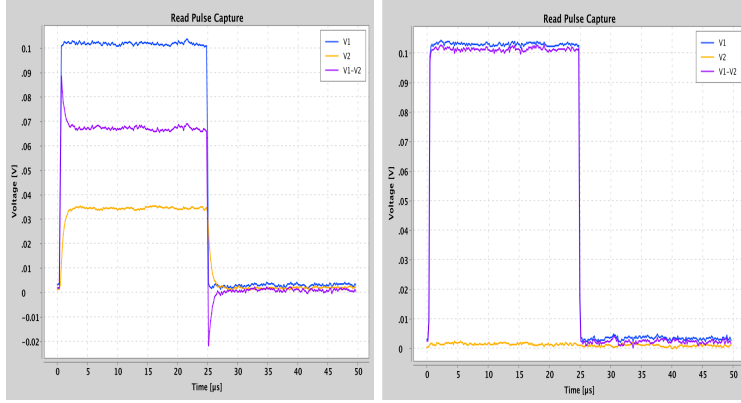
to 1 second by default. The measured conductance of the memristor over time will be displayed in the lower plot. These measurements will continue until the 'Stop' button (previously the 'Start' button) is pressed.

You can select three plots to display in the main plot area via the radio button at the bottom of the screen, below the conductance-vs-time plot. 'Write-Erase Pulse Capture' displays V1 (Blue), V2(Yellow) and V1-V2(Purple), which is the voltage drop across the memristor. 'Read Pulse Capture' displays the same voltages, but will update for every new read pulse. The conductance of the memristor is calculated from this information. Due to RC effects, it is important to understand how this occurs and helpful to see the raw data to make sure you are not operating outside the optimal parameters for measurement. We will discuss this in more detail below. Finally, the 'I-T' plot converts the data gathered in the 'Write-Erase Pulse Capture' data and, using the value of the series resistor, computes the current.

The pulse can be applied again by clicking 'Stop' and then 'Start' again or by pressing the 'S' key on the keyboard to toggle between the two. Each time the 'Start' button is pressed, a pulse will be generated, waveforms captured, and measurement pulses will commence. While the main pulse capture plots will update, the Conductance vs Time plot will simply add the new measurements and grow in size. If you would like to reset all plots, select the Pulse experiment from the main menu.

8.2 Read Pulse Measurement

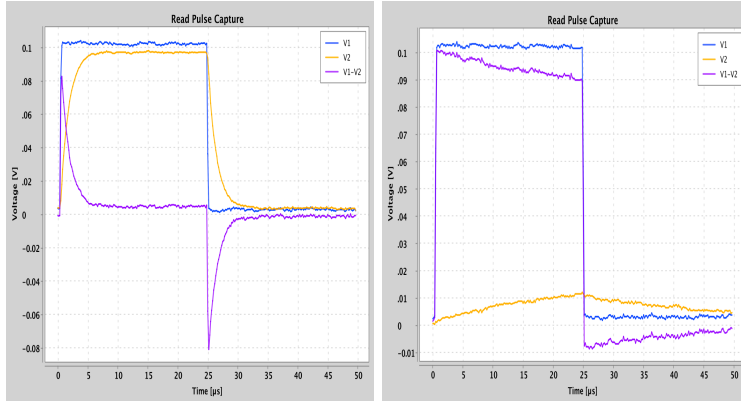
As mentioned, the conductance of the memristor is computed from the read pulse capture data. This data looks like Figure 28 using a V0 or V1 board measuring a $9.85\text{k}\Omega$ and $1\text{M}\Omega$ memristor value with a $5\text{k}\Omega$ series resistor. Notice two things. First, for the $9.85\text{k}\Omega$ case, we can see that the voltage across the series resistor settles very quickly on the value you would expect for a voltage divider: $V_2 = V_1 * \frac{R_s}{R_m + R_s}$. For .1V read pulse, this is .034V. We could therefore compute the current using this voltage drop and use the current to compute the resistance of the memristor. Indeed, this works great for low resistance values. However, if the resistance of the memristor is very high, then there is no detectable voltage drop across the series resistor if it is small.



(a) $10\text{k}\Omega$ memristor resistance, (b) $1\text{M}\Omega$ memristor resistance,
 $5\text{k}\Omega$ series resistor $5\text{k}\Omega$ series resistor

Figure 28: Read pulse capture data for low and high memristor resistance using a $5\text{k}\Omega$ series resistor.

To increase the accuracy of our measurements for higher resistance values, we could use a larger series resistor. However, if we do this, we get another problematic result, which we show in Figure 29.



(a) $10\text{k}\Omega$ memristor resistance, (b) $1\text{M}\Omega$ memristor resistance,
 $216\text{k}\Omega$ series resistor $216\text{k}\Omega$ series resistor

Figure 29: Read pulse capture data for low and high memristor resistance using a $216\text{k}\Omega$ series resistor.

While the $9.8\text{k}\Omega$ case stabilizes within about $5\mu\text{s}$, the $1\text{M}\Omega$ case does not. Due to parasitic capacitance, the $25\mu\text{s}$ period is not sufficient for the voltage to reach a stable state. We therefore cannot simply measure the voltage drop across the series resistor to determine the current as we did before.

To account for the parasitic capacitance of the board, as well as the 'parasitic' $1\text{M}\Omega$ resistance of the 1X scope of the AD2, we have integrated our open-source Spice circuit simulator JSpice into Memristor discovery. The simulation engine performs a temporal simulation of the board with its parasitic capacitance, 1X scope resistance, and the given series resistor (Figure 30) for many values of memristor resistance, from 100Ω to $10\text{M}\Omega$. It records the final voltage at the end of the read pulse time window and then creates a look-up table that associates this voltage with a memristor resistor. In this way, we are able to get fairly accurate readings of resistance even with various board parasitics. All of this happens in less than a second when you open up the experiment or when you change the series resistor value.

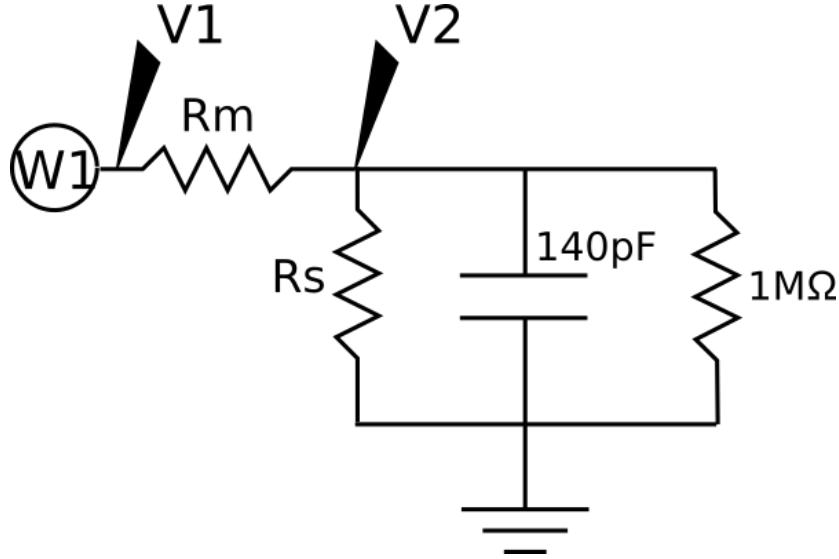


Figure 30: Equivalent circuit of Pulse experiment including parasitics using version 1 or 2 board. R_m (Memristor Resistance), R_s (Series resistor), parasitic capacitance (140pF) and 1X scope resistance ($1\text{M}\Omega$)

To achieve the most accurate measurements, monitor the read pulse capture plot. If the V_2 voltage is too close to $.1\text{V}$ or 0V , you might want to adjust your series resistor to be closer in value to the memristor resistance. Otherwise note that the reported measurement will be inaccurate.

8.3 Polarity Inversion due to Parasitic Capacitance

Polarity inversion was a problem that affected our V_0 and V_1 boards. The V_2 Board circuit with memristor anodes grounded prevents this issue.

The board parasitic capacitance combined with large series resistance can cause unintended polarity inversions that will act to change the memristor in a way you may not be intending. If you are not mindful of this, you could be

in for a frustrating experience. This is not the fault of the memristor if the last voltage polarity the memristor sees is negative. You can see the polarity inversion in Figure 31 (a). Note that 'V1-V2', which is the voltage drop across the memristor, goes negative when the applied square pulse drops back to zero. Charge stored on the parasitic capacitance can only dissipate via the series resistor or memristor, which are both of large value. The result is a reverse polarity that can (and will!) erase the memristor. While you think the conductance of the memristor should be going up, it instead goes up...and then down.

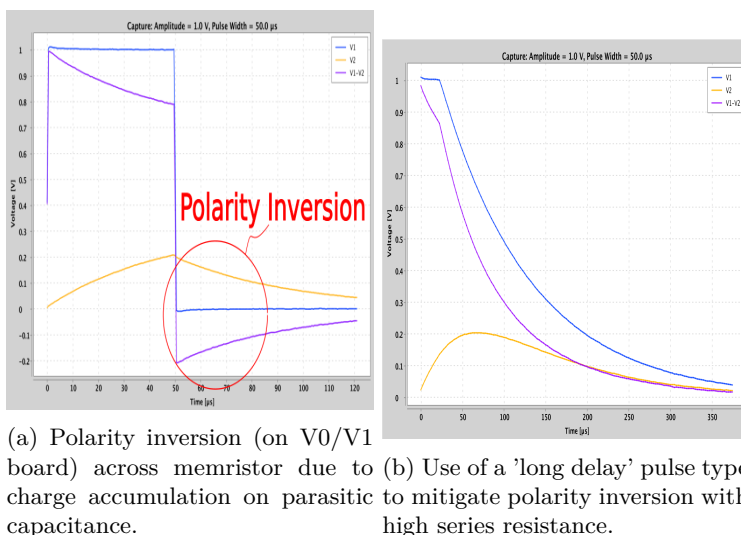


Figure 31: Polarity inversion due to high series resistance and parasitic capacitance can be mitigated by using shaped pulses.

If you would like to apply pulses with high series resistance circuits, you can still do so by using specially shaped pulses. This is shown in Figure 31 (b), where the 'SquareLongDecay' pulse is used.

8.4 Pulse Programming

Erase Write Read To measure a pulse response, choose the "Pulse" experiment. Insure that the value of the series resistor you have placed in the resistor socket matches what is stated in the 'Series R [Ohms]' field. Due to complications with parasitic capacitance, we recommend 5-20k Ω . Choose a pulse shape, for example 'SquareDecay'. Set the voltage amplitude to .5V, the pulse width to 100 μ s (the longest setting), the pulse number to 1 and the duty cycle to .5. Making sure a memristor is selected, click 'Start'. This will apply a pulse and start the measurement taking process.

Take note of the 'Write-Erase Pulse Capture' plot, in particular the V1-V2 trace. Does that trace go below zero on the falling edge of the pulse? If the

answer is yes, does the magnitude exceed $-1V$? If it does, we recommend using the 'SquareDecay' pulse type. Change the pulse type, click 'Stop' and then 'Start' again.

Take a look at the Conductance vs. Pulse Number plot, which should be updating every 1 second. The measured resistance is displayed in the plot title and as an orange line in the plot. We will first attempt to reset the device to its HRS. Set the pulse amplitude to $-2V$ and click the 'Stop/Start' button twice—first to turn off the background read measurements and then to start again and apply the new pulse. The conductance of the device should fall if it is not already in its HRS. Now set the amplitude to $+1V$, and apply the pulse again. The conductance should increase. Keep alternating the voltage amplitude between $+1V$ and $-2V$, apply a pulse and let a few read measurements pass. The memristor should settle between a HRS and a LRS. In our case, this was $200k\Omega$ and $60k\Omega$, with a $5k\Omega$ series resistor. After setting the memristor in the LRS, let the Pulse Experiment run for awhile taking measurements and observe how the conductance changes over time. You can see the result of our trial in Figure 32. We started close to the LRS.

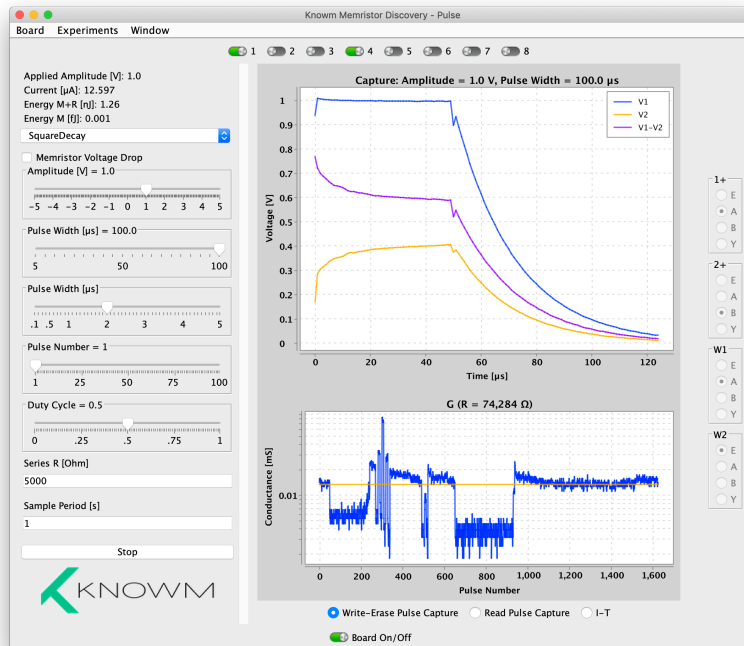


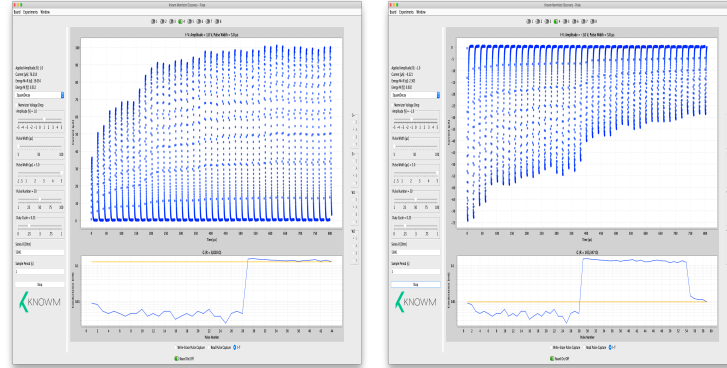
Figure 32: Example result from Pulse Experiment (V1 board),

Around pulse 20 we applied a $-2V$ pulse and drove the memristor a HRS. Starting around pulse 220 we applied alternating $+1V$ and $-2V$ pulses. Note

how the conductance of the memristor decays or relaxes over a period of a couple minutes before stabilizing. You can see this most clearly after we apply a +1V write pulse around pulse number 920. By pulse 1100 the conductance has stabilized.

Pulse Incremental Conductance Change After establishing the HRS and LRS for a long pulse width (100 μ s) at some voltage, we can reduce the pulse width to drive smaller conductances changes. Incremental resistance changes can be tricky on the MD board owing to the series resistor and parasitic capacitance. Recall that the series resistor circuit results in negative feedback for positive polarity and positive feedback for negative polarity. Since the memristor value is changing, the voltage drop across the memristor is also changing. The result is that it can be difficult to apply the same Voltage-Time impulse to the memristor as it changes resistance.

It is possible to increment a device over many hundreds of pulses as pulse widths become short. The resistance change is typically not deterministic, however. A pulse may not induce a change but another identical pulse will. You can most easily see this effect by applying not one pulse but many pulses by setting the 'Pulse Number' slider. A shorter Duty Cycle of .25 is recommended in this case. The I-T plot will show the computed current over time and this can be used to easily see the resistance increasing or decreasing as pulses are applied, as shown in Figure 33.



(a) Incremental conductance increase due to positive polarity pulses. (b) Incremental conductance decrease due to negative polarity pulses.

Figure 33: Incremental conductance changes can be most easily observed by using a pulse train and viewing the I-T plot

Due to the limitation of the Memristor Discovery board, we generally advise using longer pulse widths and lower voltages, a lower series resistor of 5-10k Ω , and attempting to operate the memristor under 75k Ω .

9 Thermodynamic Random Access Memory (kT-RAM)

Thermodynamic Random Access Memory (kT-RAM) is a new type of computing substrate. Unlike a von Neumann digital architecture where memory is physically separated from processor, kT-RAM exposes a collection of differential-pair memristive ‘synapses’ that act as both memory and processor. These synapses can be used for universal digital computation (memory and logic) but also, and perhaps more interestingly, as weighting factors in inference operations much like a neuron. While the differential-pair memristive synapse are like synapses or weights and collections of these synapses are like neurons, we must emphasize that they are also unique and offer possibilities that extend beyond what is traditionally thought possible. kT-RAM is composed of cores, where each core can be partitioned into collections of synapses of arbitrary sizes (“neurons”). Neurons can be allocated over multiple cores, single cores, all the way down to individual synapses on one core. For more information please see:

Nugent, M. Alexander, and Timothy W. Molter. “Thermodynamic-RAM technology stack.” *International Journal of Parallel, Emergent and Distributed Systems* 33.4 (2018): 430-444.

<https://arxiv.org/pdf/1406.5633.pdf>

10 Synapse Experiment

This experiment allows you to drive kT-RAM differential-pair memristor synapses with elemental kT-RAM instructions and observe a continuous response in synaptic state and synaptic pair conductances via repeated FLV read instructions. When the “Start” button is clicked or the “s” key is pressed, the selected instruction is executed, followed by continuous read instructions executed at the given sample rate. Read instructions will be read until the “Stop” button is clicked or the “s” key is pressed, at which point a new instruction can be selected and the process repeated. The pulse shape, amplitude and width can be varied.

10.1 Synapse Selection

A synapse is formed by selecting two memristors to form a differential pair. Memristor A is selected from 1–8 in the top memristor selection menu, while memristor B is selected from 8–16. Two and only two memristors can be selected. The series resistors in each of the A and B resistor sockets must match each other and the value set in the preferences menu. It is recommended to use a precision resistor to minimize measurement error.

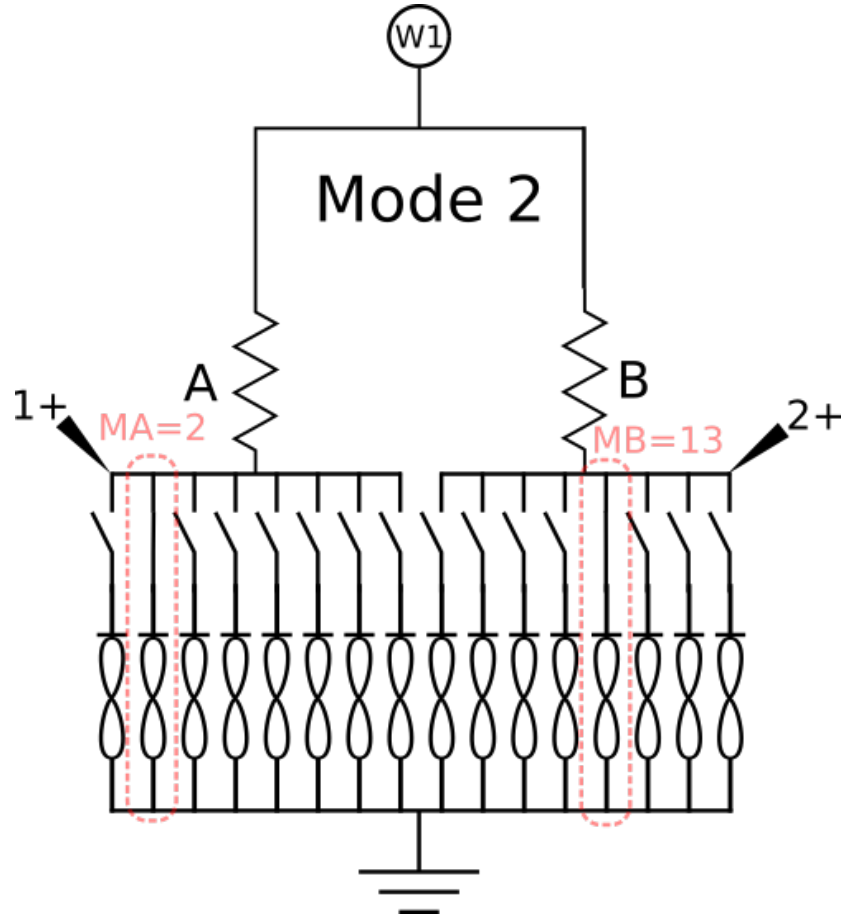


Figure 34: 1-2 Differential pair memristor synapse circuit formed from memristors 2 and 13 (of 16)

Figure 34 depicts a synapse circuit where memristor A is selected from memristor 2 on the chip, while memristor B is selected from memristor 13 on the chip. Note that all of the memristor anodes share a common ground. To increase the conductance of the selected memristors it is necessary to lower the driver voltage W1 below zero (ground). The applied pulse on W1 is negative while the voltage drop across the memristor is defined to be positive, since the cathode is the lower potential. The list of instructions is described in Table 2.

10.2 Synapse Instructions

Instruction	Description
FLV	W1=-.08V
FA	W1=- V_F , Memristor B is unselected
FB	W1=- V_F , Memristor A is unselected
FAB	W1=- V_F , Memristor A and B selected
RA	W1= V_R , Memristor B is unselected
RB	W1= V_R , Memristor A is unselected
RAB	W1= V_R , Memristor A and B selected

Table 2: 1-2 Synapse Instruction Set. V_F =Forward Amplitude, V_R =Reverse Amplitude, pulse width as selected.

10.3 Basic Use

Place the board into Mode 2 and make sure two resistors of equal value (5k Ω or greater) are placed into the A and B resistor sockets. Open preferences and insure that the value of these resistors are entered into the "series resistor" field. Place a Known 1X16 linear array memristor chip into the socket. From the menu, set the Forward and Reverse amplitudes. Keep these to less than 1 volt to start, and only increase past one volt if there is no response with higher pulse widths. Select the two memristors that will form the single differential-pair memristors. In our case we have selected memristors 2 and 13. Select the 'FLV' instruction, which stands for "Forward Low Voltage". This instruction will apply a -.08V voltage pulse of the selected pulse width on W1 and will ignore the forward voltage setting. Once the instruction has been applied, the experiment will continue to apply the FLV instruction every 1 second, or whatever value has been entered in the "Sample Rate [s]" field in the lower left portion of the control panel.

Each read operation takes the voltage measured at each of the A and B voltage divider circuits formed by a memristor and a series resistor. The synapse value, V_y , is displayed on the plot as purple. The conductance of the A and B memristors are displayed as blue and orange, respectively. The value V_y is computed as $V_y = \frac{V_1 - V_2}{V_{W1}}$, where V_1 is the voltage measured by the '1+' scope channel on the AD2 and V_2 is the voltage measured by the '2+' scope channel, as in Figure 34.

After a dozen or so read instruction have executed, select the 'FA' instruction, press 'Stop' and the 'Start'. If there is not an increase in the conductance of the A (blue) memristor, increase the pulse width and try again. If the memristor displayed the characteristic IV trace in the hysteresis or DC experiments, it should work here. The most common issue is too low a pulse width and/or too low an applied voltage amplitude. Go ahead and cycle through 5 or 10 clicks of the 'Start/Stop' button. Each cycle will result in the execution of the instruction. Next select the 'FB' instruction, click 'Stop' and the 'Start' again.

Follow this with the 'RA' and then 'RB' instruction. You should see something like Figure 35.

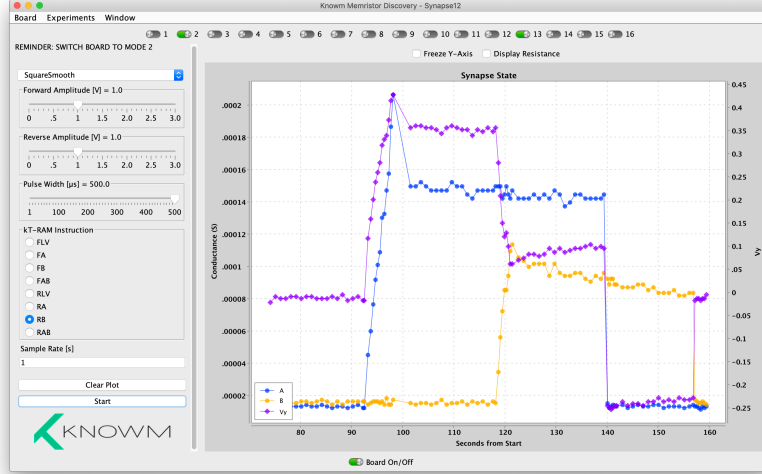


Figure 35: The result of executing about roughly ten (10) each of the FA, FB, RA and RB instructions

By adjusting the applied voltage and pulse width it is possible to incrementally drive the state of the synapse. One can think of synapse as a single bit, multi-state memory, or more generally as a weighting factor in a larger arrays of synapses. We will explore this idea in the next experiment.

11 Classifier Experiment

The Classifier Experiment allows us to execute a few simple 'kT-RAM Routines' on an 8 synapse 'neuron' formed from the 16 memristors of a 1X16 linear array chip. As in the Synapse experiment, we can select the forward and reverse pulse voltage amplitudes and pulse width. Rather than select the memristors that form each synapse, they are hard-wired in the code as shown in Table 3.

Synapse	Memristor A	Memristor B
1	1	9
2	2	10
3	3	11
4	4	12
5	5	13
6	6	14
7	7	15
8	8	16

Table 3: Synapses formed of differential memristor pairs in the 1-2 Classifier Experiment

It is not required to select any memristors, and any selections made will be overridden by the program. Instead, we can select a dataset to train our neuron with. When the 'Learn' button is clicked, the experiment will apply the selected dataset to the neuron for as many 'Tran Epochs' as is specified in the control panel. The top plot displays an exponential moving average of the classification accuracy while the bottom plot displays the measured synaptic states after each training epoch. The synaptic states are measured by performing a sequence of FLV instructions with the specified synapse active.

11.1 kT-RAM Learn Routines

A kT-RAM routine is simple a program which executes kT-RAM instructions. Many routines are possible for both supervised and unsupervised learning. The Classifier experiment allows you to test three simple routines, which are defined in the source code (Classify12Experiment.java, line 132-160) but we include here for reference:

```
private void learnCombo(SupervisedPattern pattern, double Vy) {
    if (pattern.state) {
        KTRAM_Controller.executeInstruction(pattern, Instruction12.FA);
        KTRAM_Controller.executeInstruction(pattern, Instruction12.RB);
    } else if (Vy > 0) {
        KTRAM_Controller.executeInstruction(pattern, Instruction12.RA);
        KTRAM_Controller.executeInstruction(pattern, Instruction12.FB);
    }
}

private void learnAlways(SupervisedPattern pattern, double Vy) {
    if (pattern.state) {
        KTRAM_Controller.executeInstruction(pattern, Instruction12.FA);
        KTRAM_Controller.executeInstruction(pattern, Instruction12.RB);
    } else {
        KTRAM_Controller.executeInstruction(pattern, Instruction12.RA);
    }
}
```

```

        KTRAM_Controller.executeInstruction(pattern, Instruction12.FB);
    }
}

private void learnOnMistakes(SupervisedPattern pattern, double Vy) {
    if (Vy < 0 && pattern.state) {
        KTRAM_Controller.executeInstruction(pattern, Instruction12.FA);
        KTRAM_Controller.executeInstruction(pattern, Instruction12.RB);
    } else if (Vy > 0 && !pattern.state) {
        KTRAM_Controller.executeInstruction(pattern, Instruction12.RA);
        KTRAM_Controller.executeInstruction(pattern, Instruction12.FB);
    }
}
}

```

Listing 1: Three simple kT-RAM learn routines

In the code, a SupervisedPattern is simple a pairing of an integer set with a boolean state:

```

public class SupervisedPattern {

    public final boolean state;
    public final List<Integer> spikePattern;

    public SupervisedPattern(boolean state, List<Integer> spikePattern) {
        this.state = state;
        this.spikePattern = spikePattern;
    }
}

```

Listing 2: SupervisedPattern Java Class

The kT-RAM Controller takes the integer set of the supervised pattern and activates (enables or turns on) the corresponding synapses. ' V_y ' is the evaluation voltage from a FLV read operation. Depending on the actual (V_y) versus desired ('SupervisedPattern.state') state, the learn routine will apply an instruction. As can be appreciated there are many ways to use the instructions to learn patterns. We have spent many years previous to Memristor Discovery designing kT-RAM routines on our emulators, solving problems across the broad domain of memory, logic and machine learning—both supervised and unsupervised. Our main take-away is that this particular combination of an integer-set 'spike code' and simple instructions sets is sufficient to solve any computational problem. While the details of kT-RAM programming is outside the scope of this manual, we will simply say that much is possible and we are certain much is left to discovery.

11.2 Datasets

Each dataset is a small collection of 'Supervised Patterns' and are defined as in Table 4.

Dataset	True Patterns	False Patterns
Ortho2Pattern	[0,1,2,3]	[4,5,6,7]
AntiOrtho2Pattern	[4,5,6,7]	[0,1,2,3]
Ortho4Pattern	[4,5],[6,7]	[0,1],[2,3]
AntiOrtho4Pattern	[0,1],[2,3]	[4,5],[6,7]
Ortho8Pattern	[4],[5],[6],[7]	[0],[1],[2],[3]
AntiOrtho8Pattern	[0],[1],[2],[3]	[4],[5],[6],[7]
TwoEightPattern5Frustrated	[0,1],[1,2],[2,3],[3,4]	[4,5],[5,6],[6,7],[7,8]
TwoPattern36Frustrated	[0,1,2,3,5]	[2,4,5,6,7]

Table 4: Classifier Experiment Datasets

11.3 Scramble

The 'Scramble' button will randomize the synaptic states. The code for the kT-RAM routine can be found in Classify12Experiment.java on line 225 but we include it here for reference:

```

for (int epoch = 0; epoch < controlModel.getNumTrainEpochs();
    epoch++) {
    for (int i = 0; i < 4; i++) {
        Collections.shuffle(allSpikes);
        SupervisedPattern pattern = new SupervisedPattern(true,
            allSpikes.subList(0, 1));
        KTRAM_Controller.executeInstruction(pattern,
            Instruction12.FLV);
        if (KTRAM_Controller.getVy() >= 0) {
            KTRAM_Controller.executeInstruction(pattern,
                Instruction12.RA);
            KTRAM_Controller.executeInstruction(pattern,
                Instruction12.FB);
        } else {
            KTRAM_Controller.executeInstruction(pattern,
                Instruction12.FA);
            KTRAM_Controller.executeInstruction(pattern,
                Instruction12.RB);
        }
    }
    readAllSynapses();
}

```

Listing 3: Code executed when Scramble button is pressed

11.4 Basic Use

Place the board into Mode 2 and make sure two resistors of equal value ($5k\Omega$ or greater) are placed into the A and B resistor sockets. Open preferences and insure that the value of these resistors are entered into the "series resistor" field. Place a Knowm 1X16 linear array memristor chip into the board socket. Set the forward and reverse amplitude to 1V and the Pulse Width to 200 μ s. Select the 'Ortho8Pattern' Dataset and the 'LearnOnMistakes' kT-RAM Routine then click the 'Learn'. The 'Train Accuracy' plot should update in real time as the kT-RAM Routine is run. If the Train Accuracy does not reach 1, increase the forward and reverse voltage by increments of .1 and press 'Learn' again. If the accuracy does not go to 1 by the time you reach 2V, stop and take a look at the synaptic states to see what synapses are causing the problem. The 'Ortho8Pattern' is very simple. A perfect score is achieved if synapses 1-4 are positive and synapses 5-8 are negative. Now change the dataset to 'AntiOrtho8Pattern'. This is the opposite learning problem, so all the synaptic states should change—except perhaps any synapse that may be stuck. An example result is shown in 36.

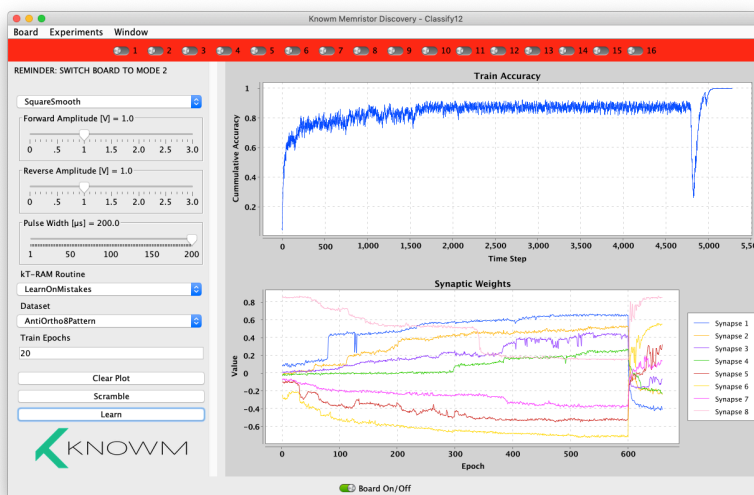


Figure 36: An example result from the Classify12 experiment

In this specific case we can see that the synapses all gradually incremented into positive and negative values. All synapses except for synapse 8 reached their target state by epoch 300. The dataset was changed to 'AntiOrtho8Pattern' around epoch 600 and the training accuracy quickly goes to 1. We can see that synapse 8 responded this time, becoming more positive. In this specific case it would be a good idea to take a careful look at Synapse 8, which is constructed from memristors 8 and 16. We can use the Synapse12 experiment to take a

closer look at the individual conductances of the each memristor. The result is shown in Figure 37, where memristor A (number 8) has been driven to a very low resistance state. Such a state can be difficult to recover from and is usually the result of ESD or driving the device too hard, i.e. higher voltages or without adequate current limiting. To recover the memristor, follow the instructions as outlined in the 'Hard Reset' section in the Important Notes section at the end of this manual.

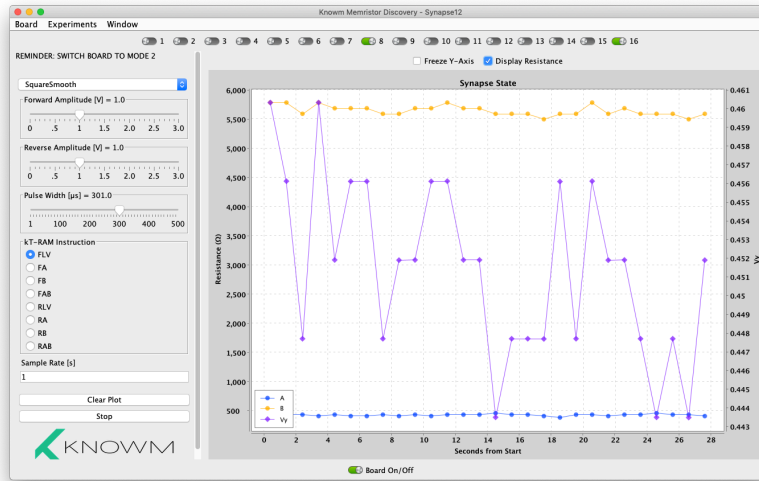


Figure 37: A 'stuck synapse' where one of the memristors has attained a very low resistance.

Once a hard reset was performed on memristor 8 to recover it's operation, we are able to learn the dataset without issues, as seen in Figure 38.

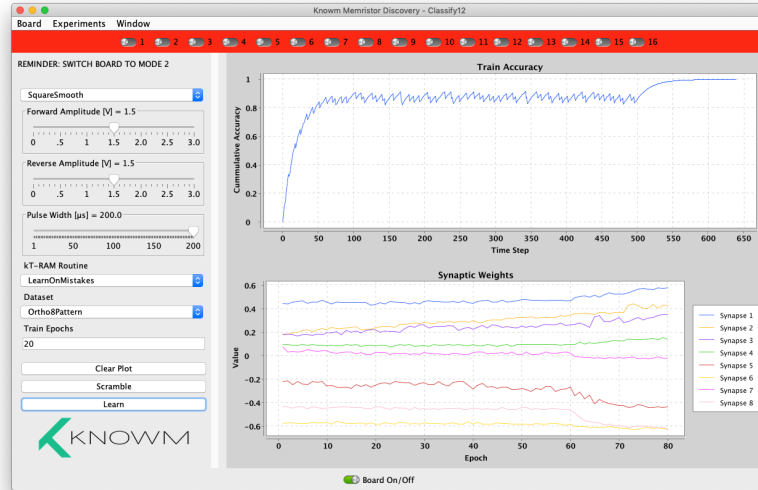


Figure 38: Result of Classifier12 experiment after memristor 'Hard Reset' recovery operation.

11.5 A Note on Voltage Drops

As we go from the 'Ortho8Pattern' dataset to a dataset where more synapses are simultaneously active, you may notice that the classifier does not learn unless the voltage is increased. This is due to the higher parallel conductance of the simultaneously active memristors. This results in a larger voltage drop across the series resistor and a correspondingly smaller voltage drop across the memristors. While it is a trivial matter to dynamically change the applied pulse magnitude, we feel it is better to leave this to the user so as to make you aware of the dynamics at play. While relatively simple, they are important.

12 Important Final Notes

12.1 Please Report Mistakes

If you discover a mistake while reading this manual, or there is something that is unclear or confusing, please let us know so that we can fix it. We provide this material so that others can learn, but we are a small team with many responsibilities. Reporting mistakes helps us to improve the material and ultimately helps everybody.

12.2 Anomalies

Almost everything about Memristor science is new. While the theory of a memristor has been around since 1970, the ability to actually use memristors is very new. M-SDC Memristors are new and the Memristor Discovery platform is new. While we have taken the initiative to put together a package for others to learn with, *we are still learning ourselves!* If you notice anomalies in the behavior of the Memristors, take note! If you can repeat the anomaly—or better—if you can describe how you got it and others can repeat it, then this when we all learn and make progress.

12.3 Glass Phase Change

While we have taken care to communicate that the W-SDC memristor are fragile devices that will be damaged if you are not careful, the reality is that they can be rather robust devices and survive a surprising amount of abuse. The problem is that a device usually does not just 'die'. Rather, it starts to function in odd ways. For example, you actually can apply significantly higher voltages than 1V and the device will switch. Depending on the duration the voltage was applied and the current through it, the chalcogenide material (a fancy name for 'glass') can melt and then freeze again. When it does this, the channels that form the pathways the Ag+ ions use to modulate conductance can radically change. This can have the effect of changing the HRS, LRS and thresholds. If you drive the device hard you can even reverse the polarity of the device by driving all the Ag+ ions to the cathode. In this state a negative polarity will cause conductance increase! Even more confusing, sometimes a device that did not work or worked poorly will work *better* after it is abused with high current. "Sometimes" is the appropriate word here. Because we do not have fine control over the applied current in the nanosecond timescale with the AD2, it is not possible to reliably control the transition between ordered and amorphous glass as is typical in phase-change materials. Consequently, it is best to avoid inducing a phase-change by being careful with applied voltage if you can.

12.4 Hard Resets

Method 1 (Try first) To recover a device that is stuck in a higher conductive state, open the the Hysteresis Experiment. Using a 10Hz signal, apply a 1V amplitude sine wave and monitor the current with the I-V plot. Slide the offset to -2V and hold it for 2-5 seconds. Return the offset to zero and let the device cycle. This forces Ag+ back to the anode and hopefully resets the device to a functional state. If this does not work, try method 2.

Method 2 (Last Resort) This method is risky, and should only be attempted as a last resort. The core problem of recovering a memristor that has achieved a very low resistance state is that the bulk of the voltage drop will occur across the series resistor instead of the memristor. This is shown clearly in

Figure 39. To provide a sufficient voltage we must remove the series resistor all together while applying a negative voltage bias. In the Hysteresis experiment, set the amplitude to '0' and the offset to '-1'. **It is very important to not apply a positive voltage, only negative!** Click 'Start' and then use a wire to briefly short the series resistor, dropping the whole **negative** voltage across the device. This will increase its resistance and hopefully recover operation. If the memristor does not respond to a shorted negative voltage above its threshold, then it has been fried.

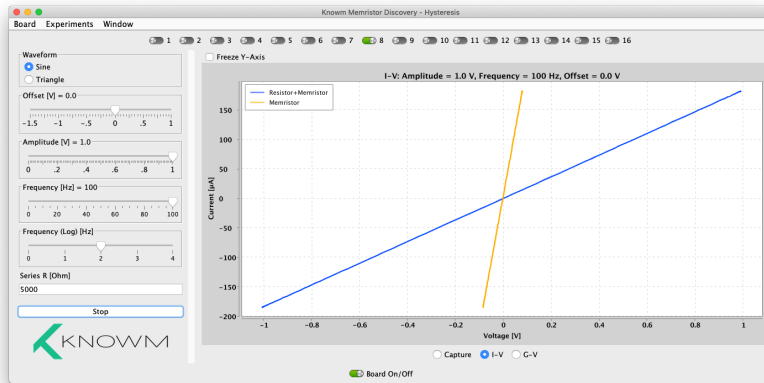


Figure 39: A very low resistance memristor in series with a current limiting resistor may not drop sufficient voltage across the memristor to clear the thresholds of adaptation. The result is a stuck memristor.

12.5 AD2 Glitches

Although rare, glitches from the AD2 have been observed that manifest in odd behavior. These are incredibly hard to debug and fix because of their rarity and inability to determine cause. Below are two glitch we are aware of or have been told about:

Glitch 1 (Severe): In one case a customer reported an intermittent failure by the AD2 to supply the proper voltage power levels. This, in turn, resulted in failures of the bilateral switches. The problem can be diagnosed with the BoardCheck experiment, although it appeared the problem disappeared/reappeared ('toggled') as experiments were changed. The best way to resolve the issue is to replace the AD2.

Glitch 2 (Communications): It appears a communication link between the AD2 and the host computer can become corrupted, resulting in odd measurements like a constant 100Ω memristor reading in the Pulse experiment, incorrect voltage offsets, and perhaps other issues. A restart of the experiment will force a new serial communications link with the AD2 and will resolve the issue.

If you are experiencing odd behavior, it may be due to your USB power supply. Try using an external 5V power supply.

12.6 AD2 Scope and Waveform Offsets

Some AD2 units have offsets of a few millivolts that can cause significant measurement issues. While the Waveforms software resolves this through calibration, and while the calibration data is stored on the device itself, the physical AD2 unit does not actually apply the calibration. Rather, Waveforms software uses the stored parameters to correct the acquired data and generated signals. To add calibration to your measurements in Memristor Discovery, follow the instructions in the "Help" Menu in versions 2.0.1 and above.